

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ CÔNG NGHỆ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ



ỨNG DỤNG TRÍ TUỆ NHÂN TẠO
TRONG TỐI ƯU TỔ HỢP
KHÓA LUẬN TỐT NGHIỆP

Giảng viên hướng dẫn: THS. NGUYỄN QUẢN BÁ HỒNG

Sinh viên thực hiện: PHAN VĨNH TIẾN

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 4 THÁNG 4 NĂM 2026

Mục lục

I	Mở đầu	4
	Danh mục viết tắt	5
1	Lý do chọn đề tài	9
2	Mục tiêu nghiên cứu	10
2.1	Mục tiêu tổng quát	10
2.2	Mục tiêu cụ thể	10
3	Đối tượng và phạm vi nghiên cứu	11
3.1	Đối tượng nghiên cứu	11
3.2	Phạm vi nghiên cứu	11
3.2.1	Phạm vi về tập dữ liệu	11
3.2.2	Phạm vi về thuật toán và phương pháp	11
3.2.3	Phạm vi về môi trường và phần cứng	12
4	Phương pháp nghiên cứu	13
4.1	Cách tiếp cận nghiên cứu	13
4.2	Quy trình thực hiện nghiên cứu	13
5	Đóng góp của khóa luận	14
6	Bố cục khóa luận	15
II	Cơ sở lý thuyết	16
7	Tối ưu tổ hợp	17
8	Bài toán người giao hàng	18
9	Tối ưu tổ hợp dựa trên mạng thần kinh và học tăng cường	20
10	Kiến trúc Transformer	21
11	Phương pháp Policy Optimization with Multiple Optima	22
12	Các nghiên cứu liên quan và khoảng trống nghiên cứu	23
12.1	Các bộ giải chính xác và heuristics	23

12.2 Sự phát triển của tối ưu tổ hợp dựa trên mạng thần kinh	24
12.3 Bài toán mở rộng quy mô	24
12.4 Kỹ thuật gộp token (ToMe) trong thị giác máy tính	25
12.5 Khoảng trống nghiên cứu	26
III Phương pháp đề xuất	27
13 Tổng quan kiến trúc	28
14 Tích hợp ToMe vào luồng đồ thị	29
15 Thuật toán	30
15.1 Thuật toán nén đồ thị bằng gộp token hai phía	30
15.1.1 Trình bày thuật toán	30
15.1.2 Giải thích các biến số	31
15.1.3 Luồng thực thi	31
15.2 Bộ mã hóa trong kiến trúc ToMe-POMO đề xuất	32
15.2.1 Trình bày thuật toán	32
15.2.2 Giải thích các biến số	32
15.2.3 Luồng thực thi	33
15.3 Huấn luyện và đánh giá Tome-POMO	34
15.3.1 Trình bày thuật toán	34
15.3.2 Giải thích các biến số	34
15.3.3 Luồng thực thi	35
16 Phân tích toán học	36
16.1 Kiến trúc tối ưu tổ hợp thần kinh dựa trên mạng thần kinh (POMO) cơ sở	36
16.1.1 Độ phức tạp của bộ mã hóa	37
16.1.2 Độ phức tạp bộ giải mã	37
16.2 Kiến trúc ToMe-POMO đề xuất	38
16.2.1 Độ phức tạp của bộ mã hóa	38
16.2.2 Độ phức tạp của bộ giải mã	39
16.2.3 Tính đẳng cự và bảo toàn không gian	39
16.2.4 Phân tích sai số	39
16.2.5 Mật độ không gian	41
17 Hàm mục tiêu và huấn luyện	42
IV Thực nghiệm và đánh giá	43
18 Thiết kế thực nghiệm	44
18.1 Môi trường và phần cứng	44
18.2 Xây dựng dữ liệu huấn luyện và kiểm thử	44
18.3 Các mô hình cơ sở để so sánh	44

19 Các chỉ số đánh giá	45
19.1 Độ lệch tối ưu	45
19.2 Thời gian suy luận	45
19.3 Thời gian huấn luyện	45
19.4 Mức tiêu thụ bộ nhớ đồ họa (VRAM)	45
20 Kết quả thực nghiệm chính	46
21 Thảo luận	49
21.1 Sự bảo toàn đặc trưng của cơ chế nén hai phía	49
21.2 Chi phí phụ trợ ở đồ thị quy mô nhỏ	49
21.3 Vai trò của độ sâu kiến trúc	49
21.4 Hạn chế của nghiên cứu và hướng phát triển trong tương lai	50
V Kết luận và định hướng	51
22 Kết luận	52
23 Định hướng	53
Tài liệu tham khảo	54

Phần I

Mở đầu

Danh mục viết tắt

Viết tắt	Tiếng Anh	Tiếng Việt
AM	Attention Model	Attention Model
CPU	Central Processing Unit	bộ xử lý trung tâm
FLOPs	Floating-point Operations Per Second	Floating-point Operations Per Second
GEMM	GEneral Matrix Multiplication	nhân ma trận tổng quát
GNNs	Graph Neural Networks	mạng neural đồ thị
GPU	Graphics Processing Unit	bộ xử lý đồ họa
KLTN		khóa luận tốt nghiệp
LKH-3	Lin-Kernighan-Helsgaun	Lin-Kernighan-Helsgaun
LSTM	Long Short-Term Memory	bộ nhớ dài hạn ngắn hạn
NCO	Neural Combinatorial Optimization	tối ưu tổ hợp thần kinh dựa trên mạng thần kinh
POMO	Neural Combinatorial Optimization	tối ưu tổ hợp thần kinh dựa trên mạng thần kinh
RNN	Recurrent Neural Network	mạng neural hồi quy
ToMe	Token Merging	gộp token
TSP	Traveling Salesman Problem	bài toán người giao hàng
ViT	Vision Transformers	Vision Transformers
VRAM	Video Random Access Memory	bộ nhớ đồ họa
VRP	Vehicle Routing Problem	bài toán định tuyến vận tải

Danh mục hình ảnh

13.1 Tổng quan kiến trúc mạng ToMe-POMO. Hệ thống tích hợp một nút thắt cổ chai không gian vào bộ mã hóa thông qua ToMe, giúp giảm thiểu độ phức tạp của các lớp Transformer sâu. Ngăn xếp hình trụ lưu trữ các hàm đóng gói để phục hồi đồ thị về nguyên trạng trước khi đưa vào bộ giải mã POMO. <i>Nguồn: Tác giả.</i>	28
14.1 Cơ chế nén và rã token hai phía. Các token được phân hoạch chẵn lẻ để thiết lập đồ thị hai phía. Thuật toán tìm kiếm các cạnh có độ tương đồng cao nhất (mũi tên đỏ) để gộp thành siêu đỉnh (màu xanh lá), đồng thời để lưu vết (nét đứt) nhằm phục vụ quá trình sao chép khôi phục ở cuối bộ mã hóa. <i>Nguồn: Tác giả.</i>	29
20.1 Biểu đồ biểu thị độ dài chu trình trên TSP-20 sau 100 epochs. <i>Nguồn: Tác giả.</i> . .	47
20.2 Biểu đồ biểu thị độ dài chu trình trên TSP-50 sau 50 epochs. <i>Nguồn: Tác giả.</i> . . .	48

Danh mục bảng biểu

20.1 Khảo sát siêu tham số trên tập TSP-20 (10 epochs). Đánh giá ảnh hưởng của tỷ lệ nén (r) và vị trí lớp nén (ℓ) đối với mô hình ToMe-POMO. Chiều dài tối ưu tuyệt đối của Concorde là 3.83 [Kwo+20]. <i>Nguồn: Tác giả.</i>	46
20.2 So sánh hiệu năng tổng thể trên TSP-20 và TSP-50. Đánh giá các biến thể ToMe-POMO tối ưu so với POMO nguyên bản sau khi huấn luyện. Độ lệch (gap) được tính tương đối so với POMO. <i>Nguồn: Tác giả.</i>	47

Kiến thức cần biết

Định nghĩa 1 (Vector). *Vector được định nghĩa là một dãy số hữu hạn được sắp xếp theo một thứ tự xác định. Về quy ước, vector ký hiệu bằng chữ cái thường đậm, các thành phần ký hiệu bằng chữ cái thường kèm chỉ số. Ví dụ $\mathbf{x}$ và các thành phần là $x_1, x_2, \dots, x_n$. Đối với vector có các thành phần thuộc trường $\mathbb{F}$, ta viết $\mathbf{x} \in \mathbb{F}^n$. Ký hiệu vector:*

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}.$$

Định nghĩa 2 (Ma trận). *Ma trận được định nghĩa là một mảng số hai chiều, trong đó mỗi phần tử được xác định bởi hai chỉ số đại diện cho vị trí hàng và cột. Về quy ước, ma trận ký hiệu bằng chữ cái in đậm, ví dụ $\mathbf{A}$. Một ma trận có kích thước m hàng và n cột, chứa các thành phần thuộc trường $\mathbb{F}$, ta viết $\mathbf{A} \in M_{m \times n}(\mathbb{F})$. Phần tử nằm ở vị trí hàng i , cột j ký hiệu là $\mathbf{A}_{i,j}$. Ký hiệu ma trận:*

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} & \cdots & \mathbf{A}_{1,n} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} & \cdots & \mathbf{A}_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{m,1} & \mathbf{A}_{m,2} & \cdots & \mathbf{A}_{m,n} \end{pmatrix}.$$

Định nghĩa 3 (Tensor). *Trong nhiều trường hợp, chúng ta cần sử dụng những mảng số có cấu trúc phức tạp hơn hai chiều. Một mảng các con số được sắp xếp trên một lưới đa chiều với số lượng trục biến thiên được gọi chung là *tensor*. Về quy ước, tensor ký hiệu bằng chữ cái hoa đậm $\mathcal{A}$. Một tensor có d chiều với các cạnh $n_1, n_2, \dots, n_d$ thuộc trường $\mathbb{F}$, ta viết $\mathcal{A} \in T_{n_1 \times n_2 \times \dots \times n_d}(\mathbb{F})$. Để xác định một phần tử cụ thể trong tensor $\mathcal{A}$, ta sử dụng các chỉ số tương ứng với tọa độ của nó trên các trục. Ví dụ $\mathcal{A}_{i,j,k}$ đại diện cho phần tử tại vị trí (i, j, k) trong một tensor ba chiều.*

Định nghĩa 4 (Hoán vị). *Một hoán vị của tập $\{1, 2, \dots, n\}$ là một song ánh từ tập này vào chính nó. Thông thường ký hiệu một hoán vị π bởi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ là một cách xếp thứ tự của $\{1, 2, \dots, n\}$.*

Chương 1

Lý do chọn đề tài

Trong bối cảnh toàn cầu hóa và sự bùng nổ của thương mại điện tử, các hệ thống logistics và chuỗi cung ứng hiện đại đang phải đối mặt với áp lực tối ưu hóa chưa từng có. Cốt lõi của bài toán định tuyến và vận tải chính là **bài toán người giao hàng (TSP)** [App+06]. Ở quy mô công nghiệp, mạng lưới phân phối không chỉ dừng lại ở vài chục hay vài trăm điểm nút, mà thường xuyên mở rộng lên tới hàng chục nghìn, thậm chí hàng triệu điểm giao hàng. Về mặt lý thuyết độ phức tạp tính toán, **TSP** là một bài toán NP-khó [App+06]. Sự bùng nổ tổ hợp của không gian nghiệm khiến các thuật toán giải quyết chính xác như Concorde [App+06] là bất khả thi hoàn toàn về mặt thời gian khi kích thước đồ thị tăng lên. Trong khi đó, các thuật toán heuristic và meta-heuristic truyền thống dù đạt được hiệu năng xấp xỉ tốt, nhưng lại phụ thuộc quá mức vào các quy tắc được thiết kế thủ công, đòi hỏi chi phí tính toán lớn và thiếu khả năng thích ứng linh hoạt với các biến thể dữ liệu mới [Pop+24].

Sự phát triển của **tối ưu tổ hợp thần kinh dựa trên mạng thần kinh (NCO)** [Bel+16] trong thập kỷ qua đã mở ra một góc nhìn hoàn toàn mới. Thay vì thiết kế các quy tắc tìm kiếm thủ công, **NCO** sử dụng học sâu và học tăng cường để huấn luyện các mạng neural tự động “học” các chiến lược giải quyết trực tiếp từ cấu trúc dữ liệu [Bel+16]. Đặc biệt, sự ra đời của các mô hình dựa trên kiến trúc Transformer [Vas+17], điển hình như **POMO** [Kwo+20], đã thiết lập các mốc state-of-the-art mới cho bài toán định tuyến. Với cơ chế self-attention, Transformer có khả năng nắm bắt trọn vẹn các mối quan hệ topo toàn cục giữa các đỉnh trên đồ thị, từ đó tạo nên những hoán vị chu trình với độ chính xác tiệm cận các bộ giải chuyên dụng [Kwo+20]. Tuy nhiên, rào cản của **NCO** nằm ở nút thắt cổ chai về tài nguyên phần cứng. Bản chất của cơ chế self-attention đòi hỏi việc tính toán ma trận tương quan giữa mọi cặp đỉnh, dẫn đến độ phức tạp về cả thời gian và không gian bộ nhớ tăng theo cấp số nhân. Khi số lượng đỉnh của đồ thị quá lớn (chẳng hạn $N \geq 10000$), bộ nhớ yêu cầu vượt xa khả năng đáp ứng của các dòng GPU thương mại cao cấp nhất [Kor+23]. Trong thực tiễn triển khai, các hệ thống tính toán biên hay các thiết bị di động thường là những hệ thống bị giới hạn nghiêm ngặt về tài nguyên. Việc đưa một mô hình Transformer công kênh với độ phức tạp $O(N^2)$ lên các thiết bị này là điều hoàn toàn bất khả thi.

Trong thời gian qua, nhiều nỗ lực giảm thiểu độ phức tạp, chẳng hạn sparse-attention [Roy+21] đã phần nào giảm tải được bộ nhớ, nhưng lại phải đánh đổi bằng việc cắt đứt các liên kết toàn cục, dẫn đến sự suy giảm nghiêm trọng về chất lượng nghiệm [Pop+24]. Khoảng trống nghiên cứu này đòi hỏi một cách tiếp cận mang tính cấu trúc hơn. Nhìn sang lĩnh vực thị giác máy tính, kỹ thuật **ToMe** được phát triển cho **Vision Transformers (ViT)** [Bol+23] đã chứng minh được khả năng giảm thiểu tính toán bằng cách hợp nhất các token mang thông tin dư thừa. Gần tương tự với hình ảnh trong thị giác máy tính, trong các mạng lưới TSP quy mô lớn, các cụm đỉnh nằm gần nhau về mặt không gian thường có vai trò tương đồng trong cấu trúc lộ trình tổng thể. Việc liên tục tính toán attention riêng biệt cho từng đỉnh lân cận này tạo ra sự dư thừa thông tin khổng lồ. Nắm bắt được tính chất này, khóa luận đi sâu vào việc nghiên cứu và phát triển một khung giải thuật hướng tới việc tích hợp thuật toán ghép cặp đồ thị của **ToMe** vào bên trong các khối mã hóa của Transformer. Bằng cách gộp linh hoạt các đỉnh có độ tương đồng cao trong không gian nhúng, cơ chế này giúp cắt tía độ dài chuỗi token qua từng lớp của mạng, làm giảm kích thước của ma trận attention và giúp giảm độ phức tạp tính toán. Nghiên cứu là tiền đề cho các cải tiến cho phép triển khai sức mạnh của các mô hình **NCO** thế hệ mới lên các thiết bị hạn chế tài nguyên, giải quyết triệt để bài toán VRAM mà vẫn duy trì được năng lực tối ưu hóa toàn cục.

Chương 2

Mục tiêu nghiên cứu

2.1 Mục tiêu tổng quát

Mục tiêu chính của đề tài là xây dựng và đánh giá một mô hình **NCO** có khả năng giải quyết hiệu quả **TSP** ở quy mô đồ thị lớn. Cụ thể, nghiên cứu tiến hành tích hợp kỹ thuật **ToMe** vào kiến trúc Transformer của mô hình cơ sở **POMO**. Hướng đi này nhằm mục đích cắt giảm lượng bộ nhớ **VRAM** và thời gian tính toán lớn sinh ra từ cơ chế self-attention, qua đó chứng minh tính khả thi của việc triển khai các mô hình định tuyến học sâu trên những thiết bị máy tính có giới hạn về tài nguyên bộ nhớ, nhưng vẫn đảm bảo được chất lượng của chu trình tìm được.

2.2 Mục tiêu cụ thể

Thứ nhất, nghiên cứu sẽ hệ thống hóa cơ sở lý thuyết qua việc nghiên cứu và tổng hợp các kiến thức nền tảng về **TSP**, phương pháp tiếp cận **NCO** thông qua học tăng cường, kiến trúc Transformer trong xử lý đồ thị, và cơ chế hoạt động của thuật toán **ToMe**. Việc này giúp xây dựng nền móng lý thuyết vững chắc để lý giải tại sao việc gộp các đỉnh lại có thể giảm thiểu chi phí tính toán.

Thứ hai, thiết kế và tùy biến mô hình. Xây dựng một kiến trúc mạng neural dựa trên **POMO**. Trong đó, trọng tâm là thiết kế thuật toán ghép cặp phù hợp để tích hợp trực tiếp vào bên trong các lớp của bộ mã hóa của kiến trúc Transformer. Mục tiêu là để mạng có thể nhận diện và hợp nhất các đỉnh có đặc trưng không gian tương đồng nhau, từ đó thu hẹp dần kích thước của đồ thị qua từng bước tính toán.

Tiếp theo, cài đặt và huấn luyện. Lập trình cài đặt mô hình đề xuất bằng ngôn ngữ Python và thư viện PyTorch. Xây dựng môi trường huấn luyện bằng học tăng cường với hàm phần thưởng dựa trên tổng độ dài chu trình. Quá trình huấn luyện cần được tinh chỉnh để đảm bảo mô hình có thể hội tụ ổn định khi số lượng token bị giảm đi.

Cuối cùng, thực nghiệm và đánh giá hiệu năng. Thiết lập các kịch bản thử nghiệm trên các đồ thị có kích thước từ vừa đến lớn. Tiến hành đo lường và so sánh ba chỉ số cốt lõi: mức tiêu thụ **VRAM** lớn nhất, thời gian chạy thuật toán và chất lượng chu trình (đo bằng độ dài đường đi) so với các phương pháp tiêu chuẩn hiện nay như thuật toán Concorde, thuật toán heuristic Lin-Kernighan [LK73] và mô hình **POMO** gốc.

Chương 3

Đối tượng và phạm vi nghiên cứu

Contents

2.1	Mục tiêu tổng quát	10
2.2	Mục tiêu cụ thể	10

3.1 Đối tượng nghiên cứu

Dựa trên mục tiêu đã xác định, đề tài tập trung vào các đối tượng nghiên cứu cốt lõi sau:

1. Về mặt bài toán tối ưu tổ hợp: TSP đối xứng trên mặt phẳng Euclid hai chiều, nơi khoảng cách giữa các đỉnh được tính bằng khoảng cách hình học thông thường.
2. Về mặt mô hình học máy: Các cấu trúc mạng neural dựa trên cơ chế attention (đặc biệt là mô hình POMO) và thuật toán học tăng cường được ứng dụng để huấn luyện mô hình giải quyết bài toán.
3. Về mặt kỹ thuật tối ưu hóa tài nguyên: Kỹ thuật ToMe giúp giảm thiểu số lượng đặc trưng cần xử lý thông qua việc tính toán độ tương đồng cosine, vốn ban đầu được dùng trong xử lý ảnh nhưng nghiên cứu và áp dụng sang cấu trúc đồ thị.

3.2 Phạm vi nghiên cứu

Để đảm bảo tính khả thi, tập trung chuyên sâu và phù hợp với khuôn khổ thời gian thực hiện khoảng 10 tuần của một khóa luận tốt nghiệp cử nhân, đề tài được giới hạn trong các phạm vi bên dưới.

3.2.1 Phạm vi về tập dữ liệu

1. Nghiên cứu chỉ sử dụng tập dữ liệu được sinh ngẫu nhiên. Tọa độ của các điểm thành phố được tạo tự động bằng cách lấy mẫu ngẫu nhiên từ phân phối đều trong không gian giới hạn $[0, 1] \times [0, 1]$. Đề tài chưa sử dụng dữ liệu bản đồ đường bộ thực tế để tránh sự phức tạp của các ràng buộc giao thông vật lý.
2. Quy mô của đồ thị trong quá trình kiểm thử được giới hạn ở các đồ thị lớn, với số lượng đỉnh N dao động từ 100 đến 1000 đỉnh.

3.2.2 Phạm vi về thuật toán và phương pháp

1. Đề tài sử dụng phương pháp học tăng cường (không cần dữ liệu nhãn có sẵn) thay vì học có giám sát. Điều này có nghĩa là mô hình sẽ tự học cách tìm đường đi tốt nhất thông qua quá trình thử sai và cập nhật trọng số dựa trên độ dài đường đi, thay vì phải bắt chước theo kết quả của một bộ giải chính xác.

2. Nghiên cứu tập trung giải quyết bài toán **TSP** tĩnh. Nghĩa là toàn bộ tập hợp các điểm cần đến đều đã được biết trước khi thuật toán bắt đầu chạy. Đề tài chưa xét đến các biến thể động khi có điểm giao hàng xuất hiện thêm trong thời gian thực, hay các ràng buộc phức tạp như tải trọng xe và khung giờ giao hàng quy định.

3.2.3 Phạm vi về môi trường và phần cứng

Việc huấn luyện và đánh giá mô hình được thực hiện trên môi trường Google Colab với phần cứng **bộ xử lý đồ họa (GPU)** phổ thông. Đây cũng chính là giới hạn thiết thực để chứng minh rằng kiến trúc mới có thể chạy được trên các hệ thống có tài nguyên hạn chế.

Chương 4

Phương pháp nghiên cứu

Contents

3.1	Đối tượng nghiên cứu	11
3.2	Phạm vi nghiên cứu	11
3.2.1	Phạm vi về tập dữ liệu	11
3.2.2	Phạm vi về thuật toán và phương pháp	11
3.2.3	Phạm vi về môi trường và phần cứng	12

4.1 Cách tiếp cận nghiên cứu

Đề tài được thực hiện dựa trên phương pháp nghiên cứu thực nghiệm định lượng kết hợp với mô phỏng tính toán. Mô hình được vận hành dựa trên nền tảng học tăng cường, cho phép mạng neural tự động tinh chỉnh trọng số thông qua quá trình tương tác với các đồ thị sinh ngẫu nhiên, từ đó tối ưu hóa hàm mục tiêu dựa trên tín hiệu phản hồi là tổng độ dài của chu trình.

Nghiên cứu áp dụng cách tiếp cận kế thừa và cải tiến. Đề tài sử dụng trực tiếp kiến trúc mạng và mã nguồn thư viện của mô hình [POMO](#) làm cơ sở đối sánh. Từ đó, tiến hành can thiệp sâu vào các khối mã hóa bằng cách thêm vào thuật toán [ToMe](#) nhằm giải quyết bài toán tràn bộ nhớ khi mở rộng quy mô đồ thị.

4.2 Quy trình thực hiện nghiên cứu

Quy trình thực hiện nghiên cứu được thiết kế một cách tuần tự và chặt chẽ, tập trung chủ yếu vào việc thử nghiệm và đối sánh trực tiếp trên môi trường tính toán đám mây Google Colab. Cùng với đó là chứng minh toán học chặt chẽ cho độ phức tạp thuật toán cũng như một vài tính chất khác, nhằm khẳng định tính khả thi cũng như hiệu suất tối ưu của phương pháp đề xuất trên phương diện lý thuyết.

Giai đoạn đầu tiên khởi đầu bằng việc thiết lập môi trường và cấu trúc mô hình cơ sở. Cụ thể, không gian lập trình Python cùng thư viện PyTorch được khởi tạo trên nền tảng Colab, làm bộ phóng để tích hợp mã nguồn nguyên bản của kiến trúc [POMO](#) từ các kho lưu trữ mở. Cùng lúc đó, các hàm chức năng hỗ trợ cũng được lập trình để tự động sinh ra các đồ thị ngẫu nhiên, sẵn sàng cung cấp dữ liệu đầu vào cho quá trình huấn luyện và kiểm thử.

Chương 5

Đóng góp của khóa luận

Nhằm giải quyết vấn đề độ phức tạp không gian của các mạng neural Transformer trong tối ưu tổ hợp, khóa luận này mang đến ba đóng góp chính trên cả phương diện kiến trúc, nền tảng toán học và thực nghiệm:

1. Đóng góp về mặt kiến trúc và thuật toán: Khóa luận lần đầu tiên đề xuất hệ thống ToMe-POMO, tích hợp thuật toán gộp token hai phía vào bên trong bộ mã hóa của bài toán định tuyến. Điểm đột phá về mặt thuật toán là việc thiết kế cơ chế lưu vết và rã token dựa trên hàm bao đóng (closure). Cơ chế này cho phép luồng dữ liệu trung gian lách qua các lớp attention với độ phức tạp giảm đi, nhưng vẫn phục hồi vẹn nguyên cấu trúc $\mathbb{R}^{N \times d}$ ở đầu ra, đảm bảo tính tương thích tuyệt đối với phương pháp giải mã đa khởi điểm của POMO.
2. Đóng góp về nền tảng toán học: Khóa luận không chỉ dừng lại ở việc thiết kế mô hình mà còn thiết lập các định lý chặt chẽ để định lượng độ phức tạp tiệm cận cho từng thành phần của mạng. Nghiên cứu đã chứng minh bằng giải tích về sai số nội suy cục bộ, đồng thời nhận xét về mật độ không gian, chứng minh rằng sự hao hụt thông tin do phép gộp đồ thị sẽ tiệm cận về không khi kích thước bài toán tiến ra ngày càng lớn.
3. Đóng góp về thực nghiệm: Thông qua quá trình tự huấn luyện bằng học tăng cường, các kết quả thực nghiệm trên tập đồ thị TSP-20 và TSP-50 đã xác thực hiệu năng của hệ thống đề xuất. Mô hình ToMe-POMO với cấu hình cắt tía 30% token ở lớp biểu diễn sâu đã có ở mức xấp xỉ 1% so với POMO gốc, đồng thời cải thiện tốc độ suy luận. Kết quả này xác nhận tiềm năng to lớn của việc triển khai các bộ giải NCO quy mô lớn trên các phần cứng có giới hạn nghiêm ngặt về tài nguyên.

Chương 6

Bố cục khóa luận

Để giải quyết các mục tiêu đã đề ra, nội dung của khóa luận được tổ chức thành 5 phần chính, bao gồm 23 chương như sau:

Phần I: Mở đầu (từ chương 1 đến chương 6). Trình bày bối cảnh công nghiệp và học thuật dẫn đến lý do chọn đề tài. Phần này xác định rõ mục tiêu tổng quát, phạm vi giới hạn của dữ liệu và phần cứng, phương pháp nghiên cứu được sử dụng, cũng như tổng hợp các đóng góp khoa học cốt lõi của khóa luận.

Phần II: Cơ sở lý thuyết (từ chương 7 đến chương 12). Cung cấp các kiến thức nền tảng về tối ưu tổ hợp và bài toán TSP. Phân tích sự phát triển của NCO qua mạng Transformer và phương pháp học tăng cường POMO. Đặc biệt, phân tích chi tiết các nghiên cứu liên quan, giới hạn về sự bùng nổ tài nguyên, và cơ chế ToMe trong thị giác máy tính nhằm làm rõ khoảng trống nghiên cứu.

Phần III: Phương pháp đề xuất (từ chương 13 đến chương 17). Trọng tâm khoa học của đề tài. Phần này trình bày chi tiết kiến trúc ToMe-POMO đề xuất, bao gồm luồng thực thi thuật toán nén/rã đồ thị và mã giả chi tiết. Cung cấp các phân tích toán học chuyên sâu về độ phức tạp bộ nhớ, sai số nội suy, đánh giá mật độ. Cuối cùng trình bày hàm mục tiêu huấn luyện.

Phần IV: Thực nghiệm và đánh giá (từ chương 18 đến chương 21). Mô tả môi trường phần cứng, phương pháp sinh dữ liệu và thiết lập các chỉ số đánh giá (gap, inference, VRAM). Báo cáo các kết quả khảo sát siêu tham số và so sánh hiệu năng tổng thể. Đi kèm với đó là những thảo luận nhằm giải thích các hiện tượng thực nghiệm dựa trên cơ sở toán học.

Phần V: Kết luận và định hướng (chương 22 và chương 23). Đúc kết lại toàn bộ kết quả nghiên cứu và những rào cản đã giải quyết thành công. Từ đó, vạch ra lộ trình các hướng nghiên cứu tiềm năng trong tương lai để mở rộng năng lực của hệ thống lên các quy mô lớn hơn và các biến thể định tuyến phức tạp hơn.

Phần II

Cơ sở lý thuyết

Chương 7

Tối ưu tổ hợp

Tối ưu tổ hợp là một phân nhánh quan trọng giao thoa giữa toán học ứng dụng, vận trù học và khoa học máy tính. Mục tiêu cốt lõi của tối ưu tổ hợp là tìm kiếm một cấu trúc toán học rời rạc (chẳng hạn như một đồ thị, một tập con, hoặc một hoán vị) từ một tập hợp đếm được (hoặc hữu hạn) các nghiệm khả thi, sao cho một hàm mục tiêu cho trước đạt giá trị tối ưu (lớn nhất hoặc nhỏ nhất) [KV18].

Về mặt toán học, cấu trúc của một bài toán tối ưu tổ hợp được định nghĩa thông qua bộ bốn thành phần $(X, (S_x)_{x \in X}, c, \text{goal})$. Trong mô hình biểu diễn này, X đại diện cho tập hợp các trường hợp của bài toán; S_x là tập hợp toàn bộ các nghiệm khả thi tương ứng với một trường hợp $x \in X$; c là hàm mục tiêu đóng vai trò lượng hóa chi phí hoặc phần thưởng của từng nghiệm; và $\text{goal} \in \{\min, \max\}$ xác định định hướng tối ưu hóa đi tìm cực tiểu hay cực đại. Đích đến cuối cùng của bài toán là tìm ra một nghiệm tối ưu toàn cục, đạt giá trị tối ưu tuyệt đối ký hiệu là $\text{OPT}(x)$ [KV18].

Thách thức lớn nhất trong lĩnh vực tối ưu tổ hợp nằm ở việc kích thước của không gian nghiệm S_x hiếm khi bị giới hạn ở mức nhỏ, mà thường xảy ra hiện tượng bùng nổ tổ hợp theo hàm mũ hoặc hàm giai thừa. Khối lượng tính toán khổng lồ làm cho các phương pháp thông thường (chẳng hạn vét cạn) hoàn toàn bất khả thi; ngay cả những hệ thống siêu máy tính hiện đại nhất cũng sẽ bị áp đảo về mặt thời gian nếu duyệt qua mọi đường đi. Do đó, việc giải quyết các bài toán này đòi hỏi việc thiết kế các thuật toán thông minh nhằm khai thác sâu vào cấu trúc toán học đặc thù của từng bài toán.

Khó khăn hơn khi phần lớn các bài toán tối ưu tổ hợp mang giá trị ứng dụng cao đều được phân loại vào lớp bài toán NP-khó. Dưới góc độ lý thuyết độ phức tạp tính toán, việc một bài toán thuộc lớp NP-khó đồng nghĩa với việc cực kỳ khó có khả năng tồn tại một thuật toán thời gian đa thức luôn đảm bảo tìm được nghiệm tối ưu toàn cục cho bài toán [KV18].

Để giải quyết những khó khăn nêu trên, việc nghiên cứu tối ưu tổ hợp buộc phải chấp nhận thay đổi việc tìm kiếm nghiệm tối ưu toàn cục sang việc chấp nhận các nghiệm xấp xỉ với chất lượng khả thi. Từ đó, trọng tâm nghiên cứu được đặt vào việc phát triển các thuật toán xấp xỉ. Các thuật toán này đảm bảo rằng nghiệm đầu ra luôn nằm trong một giới hạn sai số tỷ lệ phần trăm nhất định so với giá trị $\text{OPT}(x)$. Bên cạnh đó, đối với những hệ thống dữ liệu khổng lồ mà ở đó thời gian thực thi là yếu tố tiên quyết, các thuật toán heuristic và meta-heuristic đóng vai trò cốt lõi. Mặc dù không đảm bảo tuyệt đối về độ chính xác lý thuyết, các phương pháp này lại mang đến những giải pháp mang tính thực tiễn cao, cho phép giải quyết hiệu quả các bài toán định tuyến, phân bổ và sắp xếp phức tạp trong thực tiễn công nghiệp.

Chương 8

Bài toán người giao hàng

Trong lý thuyết độ phức tạp tính toán và tối ưu hóa tổ hợp, **TSP** là một trong những bài toán tối ưu hóa kinh điển và được nghiên cứu sâu rộng [App+06]. Nguồn gốc của bài toán này có thể được truy nguyên từ các công trình của Sir William Rowan Hamilton và Thomas Penyngton Kirkman vào thế kỷ 19, trước khi được Karl Menger và Hassler Whitney định hình lại bài toán dưới góc độ toán học hiện đại vào những năm 1930 [Sch05]. Bài toán có thể phát biểu ngắn gọn như sau: Cho trước một danh sách các thành phố và khoảng cách (hoặc chi phí) giữa từng cặp thành phố, cần tìm lộ trình ngắn nhất đi qua mỗi thành phố chính xác một lần và quay trở lại điểm xuất phát [Sch05].

Trong lý thuyết đồ thị, **TSP** được định nghĩa trên một đồ thị đầy đủ $G = (V, E, C)$, trong đó [App+06]:

- $V = \{1, 2, \dots, N\}$ là tập hợp N đỉnh, tương ứng với N thành phố.
- $E = \{(i, j) \mid i, j \in V, i \neq j\}$ là tập hợp các cạnh.
- $C = (c_{ij})_{N \times N}$ là ma trận trọng số, với c_{ij} là chi phí di chuyển từ đỉnh i đến đỉnh j . Trong không gian Euclide 2D, đồ thị mỗi đỉnh i được biểu diễn bằng tọa độ $x_i \in \mathbb{R}^2$ và $c_{ij} = \|x_i - x_j\|_2$.

Nghiệm của bài toán là một chu trình Hamilton, được biểu diễn bằng một hoán vị $\pi = (\pi_1, \pi_2, \dots, \pi_N)$ của tập các đỉnh V . Hàm mục tiêu cần cực tiểu hóa là tổng chiều dài của chu trình [App+06]:

$$L(\pi) = \sum_{i=1}^{N-1} c_{\pi_i, \pi_{i+1}} + c_{\pi_N, \pi_1}.$$

Ngoài ra, bài toán cũng có thể được mô hình hóa chặt chẽ dưới dạng quy hoạch tuyến tính nguyên theo công thức Dantzig-Fulkerson-Johnson [DFJ54]. Cụ thể, ta cần cực tiểu hóa

$$\sum_{i=1}^N \sum_{j \neq i, j=1}^N c_{ij} x_{ij}$$

với các điều kiện ràng buộc

$$\begin{aligned} \sum_{i=1, i \neq j}^N x_{ij} &= 1, \quad \forall j \in V, \\ \sum_{j=1, j \neq i}^N x_{ij} &= 1, \quad \forall i \in V, \\ \sum_{i \in S} \sum_{j \in S, j \neq i} x_{ij} &\leq |S| - 1, \quad \forall S \subsetneq V, |S| \geq 2, \\ x_{ij} &\in \{0, 1\}, \quad \forall i, j \in V. \end{aligned}$$

Trong mô hình này, cấu trúc của chu trình được biểu diễn qua các biến quyết định nhị phân $x_{ij} \in \{0, 1\}$ cho mọi cặp đỉnh $i, j \in V$, trong đó $x_{ij} = 1$ nếu cạnh có hướng từ đỉnh i đến đỉnh j nằm trong chu trình tối ưu, và

$x_{ij} = 0$ nếu ngược lại. Mục tiêu cốt lõi của bài toán là cực tiểu hóa tổng chi phí (hoặc khoảng cách) di chuyển của toàn bộ chuyến đi, được lượng hóa bằng hàm mục tiêu $\sum_{i=1}^N \sum_{j \neq i, j=1}^N c_{ij} x_{ij}$. Để đảm bảo cấu trúc hình học của

một chu trình, mô hình thiết lập hai ràng buộc cơ bản: ràng buộc bậc vào $\sum_{i=1, i \neq j}^N x_{ij} = 1, \forall j \in V$ và ràng buộc

bậc ra $\sum_{j=1, j \neq i}^N x_{ij} = 1, \forall i \in V$. Các điều kiện này nhằm bảo đảm mỗi thành phố trên đồ thị đều được ghé thăm

đúng một lần và rời đi đúng một lần. Tuy vậy, nếu hệ thống chỉ dừng lại ở các ràng buộc bậc đỉnh, nghiệm tối ưu thu được hoàn toàn có thể bị đứt gãy thành nhiều chu trình con rời rạc thay vì tạo thành một chuyến đi xuyên suốt. Nhằm khắc phục lỗ hổng này, công thức trên bổ sung tập hợp các ràng buộc loại bỏ chu trình con $\sum_{i \in S} \sum_{j \in S, j \neq i} x_{ij} \leq |S| - 1$, áp dụng cho mọi tập hợp con $S \subsetneq V$ có kích thước $|S| \geq 2$. Bằng cách ép buộc số

lượng cạnh kết nối nội bộ bên trong bất kỳ tập con S nào cũng phải nhỏ hơn hẳn số đỉnh của chính nó, ràng buộc này loại bỏ hoàn toàn sự tồn tại của các vòng lặp khép kín cục bộ, từ đó buộc các đỉnh trong nghiệm tối ưu phải liên kết chặt chẽ thành một chu trình Hamilton duy nhất bao trùm toàn bộ đồ thị.

Chương 9

Tối ưu tổ hợp dựa trên mạng thần kinh và học tăng cường

Trước sự bùng nổ của quy mô dữ liệu, các thuật toán heuristic truyền thống (như Lin-Kernighan [LK73]) tuy nhanh nhưng yêu cầu sự tinh chỉnh thủ công và tri thức chuyên gia sâu sắc, đồng thời khó khái quát hóa cho các biến thể bài toán khác nhau. NCO ra đời như một công cụ đặc lực, trong đó sử dụng các mô hình học sâu để tự động hóa việc học các heuristic thay vì thiết kế thủ công. NCO coi việc giải TSP như một quá trình dịch chuỗi (sequence-to-sequence). Cột mốc đầu tiên của NCO là kiến trúc Pointer Networks [VFJ15]. Khác với các mô hình dịch chuỗi truyền thống bị giới hạn bởi một từ điển đầu ra cố định, Pointer Networks sử dụng cơ chế attention để trỏ trực tiếp vào các phần tử của chuỗi đầu vào. Tại mỗi bước giải mã t , mạng neural xuất ra một phân phối xác suất trên tập các đỉnh chưa được truy cập, cho phép mô hình tạo ra hoán vị của một tập hợp có kích thước thay đổi [VFJ15].

Việc huấn luyện mô hình NCO bằng học có giám sát bị gặp trở ngại lớn do chi phí tạo ra bộ dữ liệu nhân (nghiệm tối ưu thực sự của bài toán cho các đồ thị lớn) là quá đắt đỏ, đòi hỏi chạy các bộ giải chính xác như Concorde mất rất nhiều thời gian. Do đó, học tăng cường đã trở thành tiêu chuẩn cho NCO, mở đầu bởi công trình của Bello và các cộng sự [Bel+16]. Trong khung học tăng cường, TSP được mô hình hóa như một quá trình quyết định Markov. Trong đó:

- Trạng thái s_t : Bao gồm đồ thị đầu vào G và chuỗi các đỉnh đã được truy cập cho đến bước t .
- Hành động a_t : Việc chọn đỉnh tiếp theo π_t từ tập các đỉnh chưa truy cập.
- Phần thưởng R : Được định nghĩa là giá trị âm của tổng độ dài chu trình, $R = -L(\pi)$.

Mô hình đóng vai trò là một policy $p_\theta(\pi|s)$ được tham số hóa bởi θ , sinh ra xác suất của chu trình π :

$$p_\theta(\pi|s) = \prod_{t=1}^N p_\theta(\pi_t|s, \pi_{1..t-1}).$$

Huấn luyện mô hình được thực hiện bằng cách cực đại hóa phần thưởng kỳ vọng (hoặc cực tiểu hóa độ dài kỳ vọng) sử dụng thuật toán REINFORCE (dựa trên policy gradient):

$$J(\theta) = \mathbb{E}_{\pi \sim p_\theta(\cdot|s)} (L(\pi)).$$

Gradient của hàm mục tiêu theo tham số θ được xấp xỉ qua lấy mẫu Monte Carlo:

$$\nabla_\theta J(\theta) \approx \frac{1}{B} \sum_{b=1}^B \left(L(\pi^{(b)}) - b(s) \right) \nabla_\theta \log p_\theta(\pi^{(b)}|s)$$

Trong đó, B là kích thước batch, $\pi^{(b)}$ là chu trình được lấy mẫu, và $b(s)$ là giá trị cơ sở có chức năng quan trọng trong việc giảm phương sai của quá trình ước lượng gradient, giúp mô hình hội tụ ổn định hơn [Bel+16].

Chương 10

Kiến trúc Transformer

Kiến trúc Transformer, ban đầu được đề xuất bởi Vaswani và cộng sự [Vas+17] cho các bài toán xử lý ngôn ngữ tự nhiên, đã tạo ra một cuộc cách mạng trong lĩnh vực NCO. Khác với các mạng neural hồi quy xử lý dữ liệu theo tuần tự, Transformer sử dụng cơ chế self-attention để nắm bắt trực tiếp các mối quan hệ toàn cục giữa mọi phần tử trong chuỗi đầu vào. Trong bối cảnh của bài toán định tuyến như TSP, Kool và cộng sự [KVV19] đã điều chỉnh kiến trúc này thành attention Model, coi mỗi thành phố (đỉnh) là một “token” và đồ thị đầu vào là một tập hợp không có thứ tự. Mô hình bao gồm hai thành phần chính: Bộ mã hóa (encoder) và bộ giải mã (decoder). Trọng tâm tính toán nằm ở bộ mã hóa, nơi đồ thị ban đầu được nhúng thành các ma trận đặc trưng phong phú. Khi đó với N đỉnh của đồ thị, mỗi đỉnh i ban đầu được biểu diễn bằng một vector tọa độ không gian. Thông qua một phép biến đổi tuyến tính, đồ thị được ánh xạ thành ma trận nhúng ban đầu $\mathbf{H}^{(0)} \in \mathbb{R}^{N \times d}$, với d là số chiều ẩn. Bộ mã hóa bao gồm L lớp xếp chồng lên nhau. Tại mỗi lớp $\ell \in \{1, \dots, L\}$, ma trận đặc trưng $\mathbf{H}^{(\ell-1)}$ được cập nhật thông qua cơ chế multi-head attention và feed-forward network. Tính toán cốt lõi của cơ chế self-attention bắt đầu bằng việc tạo ra ba ma trận Query ($\mathbf{Q}$), Key ($\mathbf{K}$), và Value ($\mathbf{V}$) thông qua các ma trận trọng số học được $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d}$ [Vas+17]:

$$\mathbf{Q} = \mathbf{H}^{(\ell-1)} \mathbf{W}^Q, \quad \mathbf{K} = \mathbf{H}^{(\ell-1)} \mathbf{W}^K, \quad \mathbf{V} = \mathbf{H}^{(\ell-1)} \mathbf{W}^V.$$

Cơ chế scaled dot-product attention tính toán điểm số attention giữa mọi cặp đỉnh, đại diện cho mức độ tương quan giữa chúng [Vas+17]:

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Trong đó, $\sqrt{d_k}$ là hệ số tỉ lệ giúp ổn định gradient trong quá trình huấn luyện. Điểm số từ hàm softmax sẽ được nhân với ma trận $\mathbf{V}$ để tạo ra biểu diễn mới cho mỗi đỉnh, chứa đựng thông tin ngữ cảnh từ toàn bộ các đỉnh khác trên đồ thị. Để tăng cường khả năng biểu diễn, multi-head attention chia $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ thành h không gian con (heads) song song, tính toán attention độc lập trên từng không gian, sau đó nối (concatenate) kết quả lại và nhân với ma trận đầu ra $\mathbf{W}^O$ [Vas+17]:

$$\text{MHA}(\mathbf{H}) = \text{concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O.$$

Sau multi-head attention, dữ liệu đi qua lớp feed-forward network bao gồm hai phép biến đổi tuyến tính xen kẽ bởi hàm kích hoạt ReLU. Các kỹ thuật kết nối tắt (residual connection) và chuẩn hóa (batch normalization hoặc layer normalization) được áp dụng sau mỗi khối [Vas+17]:

$$\hat{\mathbf{H}}^{(\ell)} = \text{Norm}(\mathbf{H}^{(\ell-1)} + \text{MHA}(\mathbf{H}^{(\ell-1)})), \quad \mathbf{H}^{(\ell)} = \text{Norm}(\hat{\mathbf{H}}^{(\ell)} + \text{FFN}(\hat{\mathbf{H}}^{(\ell)})).$$

Mặc dù mang lại khả năng trích xuất đặc trưng vượt trội, kiến trúc Transformer đối mặt với một giới hạn nghiêm trọng khi áp dụng cho TSP kích thước lớn. Phép nhân ma trận $\mathbf{Q}\mathbf{K}^\top$ trong cơ chế attention đòi hỏi tính toán khoảng cách giữa mọi cặp đỉnh, dẫn đến độ phức tạp về VRAM và thời gian tính toán là $\mathcal{O}(N^2)$. Khi N vượt quá vài nghìn, yêu cầu tài nguyên bùng nổ, khiến việc sử dụng các cấu trúc Transformer tiêu chuẩn trở nên bất khả thi nếu không có các kỹ thuật thu gọn không gian đặc trưng [Roy+21].

Chương 11

Phương pháp Policy Optimization with Multiple Optima

Trong khung học tăng cường REINFORCE truyền thống dùng cho NCO, hàm mục tiêu cực đại hóa phần thưởng kỳ vọng đòi hỏi một giá trị cơ sở $b(s)$ để giảm phương sai của quá trình ước lượng gradient. Trước đây, giá trị này thường được xấp xỉ bởi một mạng critic độc lập [KVVW19] làm tăng độ phức tạp huấn luyện và tiêu tốn thêm bộ nhớ. Phương pháp Policy Optimization with Multiple Optima (POMO), được giới thiệu bởi Kwon và cộng sự, đã giải quyết triệt để vấn đề này bằng cách tận dụng đặc tính đối xứng của bài toán [Kwo+20]. Khởi điểm của POMO là việc nhận ra rằng: đối với bài toán TSP, một chu trình tối ưu khép kín là bất biến đối với đỉnh xuất phát. Cụ thể, một chu trình đi qua các đỉnh $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ có cùng tổng khoảng cách với chu trình $2 \rightarrow 3 \rightarrow 1 \rightarrow 2$. Tận dụng điều này, thay vì sinh ra một quỹ đạo duy nhất, POMO ép mô hình sinh ra đồng thời N chu trình khác nhau, bắt đầu từ N đỉnh phân biệt của cùng một đồ thị đầu vào s . Giả sử $\pi^{(i)}$ là chu trình được giải mã khi chọn đỉnh i làm điểm xuất phát, với $i \in \{1, 2, \dots, N\}$. Thuật toán giải mã sẽ tự động hồi quy để hoàn thiện N chu trình này song song. Độ dài (chi phí) của mỗi chu trình được tính bằng hàm $L(\pi^{(i)})$. Đột phá của POMO nằm ở việc định nghĩa lại giá trị cơ sở $b(s)$. Thay vì dùng mạng critic, POMO sử dụng chính giá trị trung bình cộng độ dài của N chu trình được sinh ra từ cùng một đồ thị làm cơ sở:

$$b(s) = \frac{1}{N} \sum_{i=1}^N L(\pi^{(i)}).$$

Lúc này, đạo hàm của hàm mục tiêu được xấp xỉ bằng phương pháp Monte Carlo trên tập hợp N quỹ đạo:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(L(\pi^{(i)}) - b(s) \right) \nabla_{\theta} \log p_{\theta}(\pi^{(i)} | s).$$

Cơ chế này mang lại cơ sở toán học cực kỳ vững chắc và hiệu quả thực tiễn cao:

1. Loại bỏ hoàn toàn mạng critic: Giảm bớt số lượng tham số cần huấn luyện, tiết kiệm bộ nhớ, và tránh được sự mất ổn định khi đồng bộ hóa giữa hai mạng actor và critic.
2. Giảm phương sai tối đa: Việc so sánh chi phí của quỹ đạo $\pi^{(i)}$ trực tiếp với mức trung bình của các quỹ đạo sinh ra từ cùng một phân phối p_{θ} trên cùng một đồ thị cung cấp một tín hiệu gradient cực kỳ sắc nét. Các quỹ đạo tốt hơn mức trung bình ($L(\pi^{(i)}) < b(s)$) sẽ có hệ số nhân âm (phần thưởng dương), qua đó xác suất sinh ra chúng được tăng cường, và ngược lại.
3. Tránh tối ưu cục bộ: Việc chủ động khám phá N không gian tìm kiếm song song buộc mô hình phải học cách xây dựng chu trình hiệu quả ở nhiều góc nhìn khác nhau, nâng cao tính tổng quát và chất lượng nghiệm cuối cùng.

Kết hợp POMO với sức mạnh biểu diễn của Transformer đã tạo ra một chuẩn mực SOTA cho các bài toán định tuyến. Tuy nhiên, việc phải duy trì N quỹ đạo song song cùng với cơ chế self-attention $\mathcal{O}(N^2)$ càng làm bài toán rò rỉ VRAM trở nên trầm trọng đối với TSP có đồ thị kích thước lớn, mở ra sự cần thiết thiết yếu của việc tích hợp các kỹ thuật giảm chiều nhúng.

Chương 12

Các nghiên cứu liên quan và khoảng trống nghiên cứu

12.1 Các bộ giải chính xác và heuristics

Trước sự ra đời của các phương pháp học sâu, việc giải quyết các bài toán tối ưu tổ hợp định tuyến như **TSP** hay **bài toán định tuyến vận tải (VRP)** bị chi phối bởi hai trường phái: các bộ giải thuật toán chính xác và các thuật toán xấp xỉ/heuristic. Hiểu rõ giới hạn của các phương pháp truyền thống này là tiền đề thiết yếu để chứng minh cho sự cần thiết của **NCO**. Đại diện tiêu biểu nhất cho trường phái bộ giải chính xác là Concorde, một hệ thống phần mềm được thiết kế bởi Applegate, Bixby, Chvátal, và Cook (2006) [App+06]. Concorde áp dụng phương pháp nhánh và cắt (branch-and-cut), tích hợp các thuật toán sinh mặt phẳng cắt (cutting plane algorithms) chuyên biệt cho **TSP** và bộ giải quy hoạch tuyến tính QSOpt. Sự vượt trội của Concorde nằm ở khả năng chứng minh tính tối ưu tuyệt đối của các lộ trình, được xác nhận thông qua việc giải quyết thành công tất cả 110 phiên bản trong bộ dữ liệu chuẩn TSPLIB, bao gồm cả phiên bản quy mô khổng lồ với 85,900 đỉnh [App+06]. Tuy nhiên, chi phí thời gian tỷ lệ thuận với hàm mũ của Concorde biến nó thành một công cụ lý thuyết hơn là thực tiễn cho các ứng dụng công nghiệp thời gian thực. Đối với các bài toán TSP với vài chục nghìn đỉnh, thời gian tính toán của Concorde có thể lên đến hàng trăm năm **bộ xử lý trung tâm (CPU)**, một giới hạn vật lý không thể chấp nhận trong các hệ thống logistics. Để đối phó với sự bất khả thi về mặt thời gian, các phương pháp xấp xỉ và thuật toán heuristic cục bộ đã trở thành tiêu chuẩn công nghiệp. Thuật toán Christofides [Chr76], dựa trên việc xây dựng cây khung nhỏ nhất (minimum spanning tree) và ghép cặp hoàn hảo chi phí cực tiểu (minimum-weight perfect matching), nổi tiếng với việc đảm bảo khoảng cách tối ưu luôn nằm trong giới hạn $3/2$ đối với TSP đối xứng tuân thủ bất đẳng thức tam giác. Tuy nhiên, trong thực tiễn, thuật toán Lin-Kernighan [LK73] và phiên bản cải tiến **Lin-Kernighan-Helsgaun (LKH-3)** [Hel17] là một trong các bộ giải tốt nhất hiện tại. **LKH-3** hoạt động dựa trên cơ chế tìm kiếm cục bộ với độ sâu biến thiên (variable depth local search). Thuật toán liên tục phá vỡ và tái cấu trúc chu trình thông qua các bước hoán đổi k -opt (k -opt submoves), thay thế k cạnh hiện tại bằng k cạnh mới để tìm kiếm độ giảm chi phí lớn nhất. Một cải tiến mang tính bản lề của **LKH-3** là việc áp dụng các hàm phạt (penalty functions) để giải quyết các ràng buộc phức tạp của **VRP** (như cửa sổ thời gian, khả năng chuyên chở) bằng cách đưa chúng trực tiếp vào hàm mục tiêu, kết hợp với kỹ thuật tìm kiếm được hướng dẫn bởi khung xương (backbone-guided search) nhằm định hướng nhiều loạn cục bộ [Hel17]. Mặc dù **LKH-3** có thể dễ dàng tạo ra các lộ trình đạt khoảng cách sai số dưới 0.1% so với mức tối ưu thực sự và có khả năng xử lý bài toán quy mô hàng triệu đỉnh, bản chất của nó vẫn là một bộ nguyên tắc thủ công cực kỳ rườm rà (hand-crafted rules). Việc thiết kế và hiệu chỉnh các hàm phạt hoặc quy tắc k -opt cho một bài toán biến thể mới đòi hỏi kiến thức chuyên môn sâu sắc về vận trù học. Hơn nữa, vì **LKH-3** phụ thuộc vào sự tìm kiếm lặp đi lặp lại từ một điểm neo khởi tạo, thời gian tính toán của nó vẫn là một nút thắt đáng kể khi môi trường phân phối yêu cầu cập nhật lại toàn bộ lộ trình liên tục theo thời gian thực [Pop+24].

12.2 Sự phát triển của tối ưu tổ hợp dựa trên mạng thần kinh

Nhận thức được rào cản về tốc độ và sự cứng nhắc của các bộ giải heuristic thủ công, cộng đồng trí tuệ nhân tạo đã mở ra hướng đi mới, chính là **NCO**. Triết lý của **NCO** là đào tạo các mô hình học sâu khả vi nhằm tự động trích xuất các mẫu đồ thị và học các heuristic định tuyến từ dữ liệu mà không cần sự can thiệp của tri thức chuyên gia. Sự tiến hóa của **NCO** được đánh dấu bởi những bước chuyển mình lớn về cấu trúc mạng và phương pháp huấn luyện. Giai đoạn nguyên thủy, các kiến trúc **NCO** sử dụng **mạng neural hồi quy (RNN)** để xử lý dữ liệu không gian, tiêu biểu là Pointer Networks do Vinyals và cộng sự (2015) [VFJ15] đề xuất, sử dụng cơ chế attention để chọn trực tiếp các token đầu vào làm đầu ra. Tuy nhiên, việc ứng dụng học có giám sát trên Pointer Networks thất bại trước rào cản nhãn dữ liệu. Việc thu thập hàng triệu nhãn lộ trình tối ưu thực sự từ Concorde để làm dữ liệu huấn luyện cho bài toán NP-khó là điều phi thực tế. Bước nhảy vọt thứ hai xuất hiện khi Bello và cộng sự (2017) [Bel+16] từ bỏ học có giám sát và đưa học tăng cường vào khung giải pháp. Bằng cách sử dụng chiều dài lộ trình âm làm tín hiệu phần thưởng, thuật toán REINFORCE kết hợp mạng actor-critic đã hiện thực hóa quá trình tự học của mạng neural. Tuy nhiên, việc nhúng không gian hai chiều thông qua **RNN** hay **bộ nhớ dài hạn ngắn hạn (LSTM)** khiến đồ thị trở nên phụ thuộc tuyến tính vào thứ tự tọa độ đầu vào, gây nhiễu loạn luồng thông tin không gian. Kiến trúc Transformer, đại diện bởi mô hình **Attention Model (AM)** của Kool và cộng sự (2019) [KVV19] đã khắc phục sự phụ thuộc thứ tự này. **AM** sử dụng kiến trúc attention-based, mô hình hóa **TSP** như một đồ thị đầy đủ. Tại mỗi bước giải mã, một “nút ngữ cảnh” tổng hợp thông tin đồ thị và trạng thái thăm hiện tại sẽ attention lên phần còn lại của không gian để xuất ra phân bố xác suất cho đỉnh tiếp theo. Kool và cộng sự cũng tối ưu hóa bộ nhớ và độ ổn định huấn luyện bằng cách loại bỏ mạng critic, thay bằng greedy rollout baseline. Tuy nhiên, vấn đề thiên vị/thiên lệch điểm neo khi sinh chuỗi tự hồi quy vẫn tồn tại. Tối ưu của mô hình hóa học tăng cường attention-based hiện tại được đánh dấu bằng phương pháp **POMO** của Kwon và cộng sự (2020) [Kwo+20]. Bằng cách kích hoạt đồng thời N quỹ đạo sinh nghiệm từ toàn bộ N đỉnh khởi tạo và đánh giá chúng qua một shared baseline, **POMO** tạo ra phương sai gradient tiệm cận 0, giúp mạng neural khám phá không gian đối xứng cực kỳ hiệu quả. **POMO** đã vươn lên thành nền tảng tiêu chuẩn cho hầu hết các giải pháp **NCO** hiện đại. Mặc dù vậy, **POMO** vẫn chịu ảnh hưởng bởi sự cạn kiệt tài nguyên vật lý khi đối mặt với quy mô bài toán hàng nghìn đỉnh [KPP22; SY23].

12.3 Bài toán mở rộng quy mô

Mở rộng quy mô (scalability) là một trong những rào cản kỹ thuật lớn nhất khi nỗ lực chuyển giao các mô hình **NCO** từ môi trường nghiên cứu lý thuyết sang các hệ thống định tuyến công nghiệp [BLP21]. Trong khi các mô hình **NCO** hiện đại dựa trên kiến trúc Transformer như attention Model [KVV19] hay **POMO** [Kwo+20] thể hiện hiệu năng xuất sắc trên các đồ thị có kích thước nhỏ và trung bình, việc áp dụng trực tiếp các kiến trúc này lên bài toán quy mô lớn với hàng chục nghìn đỉnh lại vướng phải những giới hạn vật lý của phần cứng tính toán. Sự khó khăn này không chỉ bắt nguồn từ sự bùng nổ tổ hợp của không gian nghiệm theo giai thừa của số lượng đỉnh, một trong các hệ quả tất yếu của lớp bài toán NP-khó [KV18], mà còn xuất phát từ bản chất của cơ chế biểu diễn và xử lý đồ thị bên trong mạng neural sâu. Cốt lõi của sự hạn chế tính toán này nằm ở cơ chế self-attention bên trong kiến trúc Transformer. Để mô hình có thể nắm bắt được bức tranh định tuyến toàn cục, cơ chế self-attention buộc phải tính toán mức độ tương quan chéo giữa mọi cặp đỉnh trên đồ thị thông qua phép nhân ma trận giữa tập hợp vector truy vấn (Query) và tập hợp vector khóa (Key) [Vas+17]. Phép toán QK^T này tạo ra một ma trận chú ý có kích thước $N \times N$, dẫn đến độ phức tạp về cả **VRAM** và thời gian thực thi tỷ lệ thuận theo bình phương số lượng đỉnh $\mathcal{O}(N^2)$ [Tay+22]. Khi giá trị này bị nhân lên với số lượng các lớp mạng, số lượng attention heads, và đặc biệt ví dụ số lượng quỹ đạo song song cần duy trì trong thuật toán **POMO**, tổng lượng **VRAM** yêu cầu dễ dàng bùng nổ vượt ngưỡng phần cứng thông thường [Kwo+20]. Sự tăng lên rất nhanh dữ liệu trung gian này nhanh chóng đánh bại mọi dòng **GPU** thương mại cao cấp nhất hiện nay với lỗi tràn bộ nhớ ngay trong quá trình lan truyền tiền (forward pass). Đối mặt với bài toán rò rỉ tài nguyên này, cộng đồng nghiên cứu đã đề xuất nhiều hướng tiếp cận khác nhau nhưng phần lớn đều gặp phải những sự đánh đổi về chất lượng nghiệm. Hướng tiếp cận phổ biến nhất là sử dụng cơ chế chú ý thưa (sparse attention) [Chi+19] hay nhiều cơ chế cắt tia khác, đặc biệt phổ biến trong các kiến trúc **mạng neural đồ thị (GNNs)** [JLB19]. Trong các mô hình này, mỗi thành phố chỉ được

phép tương tác với một lượng nhỏ các thành phố lân cận về mặt địa lý. Mặc dù phương pháp này có thể ép độ phức tạp tính toán xuống mức tiệm cận tuyến tính $\mathcal{O}(N \log N)$ hoặc thấp hơn, việc cắt đứt các liên kết xa khiến mô hình đánh mất hoàn toàn tầm nhìn toàn cục, dẫn đến hệ quả là mạng neural rất dễ bị mắc kẹt vào các nghiệm cục bộ (local optima) có chất lượng thấp [Tay+22]. Một hướng tiếp cận khác là chiến lược chia để trị (divide-and-conquer), tiến hành phân hoạch một đồ thị khổng lồ thành nhiều cụm đồ thị con nhỏ hơn để giải quyết riêng lẻ rồi ghép nối lại [FQZ21]. Tuy nhiên, lúc này đòi hỏi những kỹ thuật và lỗi hậu xử lý tại ranh giới của các đồ thị con rất phức tạp [Koo+22]. Đứng trước sự bế tắc của các phương pháp cắt tĩa liên kết truyền thống, bài toán mở rộng quy mô đặt ra yêu cầu cấp thiết về một cơ chế có khả năng làm giảm kích thước không gian biểu diễn của đồ thị một cách thông minh, ưu tiên giảm số lượng thực thể cần tính toán thay vì chỉ giảm bớt các cạnh kết nối, đồng thời vẫn phải bảo toàn được vẹn nguyên lượng thông tin ngữ cảnh mang tính toàn cục. Đây chính là tiền đề động lực dẫn đến việc vay mượn và tùy biến kỹ thuật ToMe, một phương pháp nén dữ liệu đột phá vốn được thiết kế cho mạng neural thị giác [Bol+23], để đưa vào ứng dụng trong bài toán tối ưu tổ hợp. Bằng cách thiết kế một thuật toán nhận diện và hợp nhất tự động các cụm đỉnh có độ tương đồng không gian cao ngay bên trong các khối mã hóa, hệ thống có thể chủ động thu hẹp chiều dài của chuỗi token đồ thị qua từng lớp của mạng neural [Bol+23]. Điều này giúp làm giảm trực tiếp kích thước của ma trận attention một cách lũy tiến, giảm bớt phần nào độ phức tạp $\mathcal{O}(N^2)$ của bộ nhớ VRAM, đồng thời các token đại diện vẫn chứa đựng lượng đặc trưng của cụm không gian được gộp lại. Hướng giải quyết mang tính kiến trúc này không chỉ mở ra khả năng thực thi thuật toán tìm kiếm đường đi trên các hệ thống thiết bị bị giới hạn về tài nguyên, mà còn là tiền đề cho các kỹ thuật mới trong việc cắt tĩa đồ thị của lĩnh vực NCO.

12.4 Kỹ thuật ToMe trong thị giác máy tính

Đối mặt với rào cản tính toán tương tự của kiến trúc ViT, các nhà nghiên cứu trong lĩnh vực thị giác máy tính đã liên tục tìm kiếm các giải pháp tối ưu. Các kỹ thuật cắt tĩa token hay bỏ qua token giúp giảm khối lượng tính toán; tuy nhiên chúng thường loại bỏ hoàn toàn các thông tin đặc trưng, dẫn đến sự suy giảm nghiêm trọng về độ chính xác do cấu trúc không gian bị đứt gãy. Để khắc phục nhược điểm này, Bolya và cộng sự đã đề xuất một phương pháp mang tính đột phá, chính là ToMe [Bol+23]. Thay vì vứt bỏ các token dư thừa, ToMe chủ động tìm kiếm và gộp các token mang thông tin ngữ nghĩa tương đồng lại với nhau một cách liên tục qua từng lớp mạng, cắt giảm trực tiếp chiều dài chuỗi token tham gia vào các phép tính $\mathcal{O}(N^2)$ của cơ chế attention mà không cần phải huấn luyện lại mô hình từ đầu. Cốt lõi thuật toán của ToMe nằm ở phương pháp ghép cặp hai phía trong lý thuyết đồ thị. Cơ chế này được tích hợp vào giữa nhánh attention và nhánh mạng neural truyền thẳng trong mỗi khối của kiến trúc Transformer. Thuật toán hoạt động thông qua một chu trình tối giản nhưng rất tối ưu trên GPU [Bol+23]:

1. Phân chia: Tập hợp các token đầu vào được chia xen kẽ thành hai tập **A** và **B** có kích thước xấp xỉ bằng nhau.
2. So sánh tương đồng: Khác với các phương pháp gom cụm (clustering) lặp đi lặp lại tốn kém như k-means, ToMe tái sử dụng trực tiếp ma trận khóa (Key) **K** từ cơ chế attention. Thuật toán tính toán độ tương đồng cosine (cosine similarity) từ mỗi token trong tập **A** đến toàn bộ token trong tập **B** dựa trên các vector khóa này.
3. Ghép cặp và gộp: Dựa trên ma trận tương đồng, thuật toán nhanh chóng xác định r cặp token giống nhau nhất. Hai token trong mỗi cặp sẽ được hợp nhất thông qua phép tính trung bình đặc trưng có trọng số, tạo ra một “siêu token/siêu nút” đại diện cho cả hai.

Bằng cách loại bỏ r token sau mỗi lớp, nếu mạng Transformer có L lớp, kích thước đầu vào ở lớp cuối cùng sẽ giảm đi $r \times L$ token, làm giảm độ phức tạp bộ nhớ từ $\mathcal{O}(N^2)$ xuống chỉ còn $\mathcal{O}((N - lr)^2)$ tại lớp thứ l . Một đóng góp toán học khác của ToMe là cơ chế proportional attention. Khi nhiều token hợp nhất, một siêu token mới sẽ chứa đựng khối lượng thông tin lớn hơn (kích thước $s \geq 2$) so với các token đơn lẻ. Để đảm bảo cơ chế self-attention không bị sai lệch do sự chênh lệch “khối lượng” này, Bolya và cộng sự đã tinh chỉnh

hàm softmax của cơ chế attention như sau [Bol+23]:

$$\text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \log \mathbf{s}\right),$$

trong đó $\mathbf{s}$ là một vector hàng lưu trữ kích thước (số lượng token nguyên thủy đã bị gộp vào) của mỗi token hiện tại. Cơ chế này đảm bảo rằng các cụm điểm không gian lớn sẽ bảo toàn được “lực hấp dẫn” của chúng trong quá trình truyền thông tin. Thực nghiệm chỉ ra rằng ToMe có thể tăng gấp đôi thông lượng tính toán cho các mô hình ViT quy mô cực lớn mà chỉ làm giảm biên độ chính xác dưới 0.3% [Bol+23].

12.5 Khoảng trống nghiên cứu

Thông qua việc tổng hợp các nghiên cứu trên, có thể nhận thấy một khoảng trống nghiên cứu rõ rệt giữa năng lực tối ưu hóa và khả năng mở rộng quy mô của các mô hình NCO hiện đại.

Cụ thể, kiến trúc POMO đã thiết lập một chuẩn mực mới về chất lượng nghiệm thông qua việc đánh giá song song N quỹ đạo độc lập [Kwo+20]. Tuy nhiên, việc duy trì N quỹ đạo này kết hợp với cơ chế self-attention nguyên bản $\mathcal{O}(N^2)$ đã khiến yêu cầu về VRAM tăng tiến rất nhanh tại bộ giải mã, gây ra lỗi tràn bộ nhớ trên các phần cứng thương mại ngay cả với các đồ thị có kích thước trung bình [Pop+24]. Để giải quyết rào cản này, các giải pháp cắt tỉa liên kết như sparse-attention dù ép được độ phức tạp xuống mức tiệm cận tuyến tính, nhưng lại làm mất đi đặc trưng toàn cục, khiến nghiệm mô hình dễ dàng rơi vào các giá trị tối ưu cục bộ.

Mặt khác, trong lĩnh vực thị giác máy tính, kỹ thuật gộp token ToMe đã chứng minh được khả năng triệt tiêu trực tiếp chiều dài chuỗi dữ liệu, tăng gấp đôi thông lượng tính toán mà không cần thiết kế lại mạng Transformer [Bol+23]. Tuy nhiên, dữ liệu hình ảnh có dung sai lớn hơn rất nhiều so với dữ liệu định tuyến. Bài toán TSP đòi hỏi cấu trúc và tọa độ không gian cực kỳ chính xác; do đó, ToMe chưa từng được tích hợp trực tiếp vào các mô hình NCO do lo ngại về sự đứt gãy thông tin biểu diễn.

Từ những phân tích trên, khoảng trống nghiên cứu cấp thiết hiện tại là việc thiếu vắng một cơ chế nén không gian đồ thị, có khả năng thu gọn trực tiếp số lượng đỉnh tham gia vào cơ chế attention trong bộ mã hóa mà vẫn bảo toàn vẹn nguyên cấu trúc đồ thị ở đầu ra để phục vụ cho các bộ giải mã phức tạp như POMO. Khóa luận này được thực hiện nhằm lấp đầy khoảng trống đó.

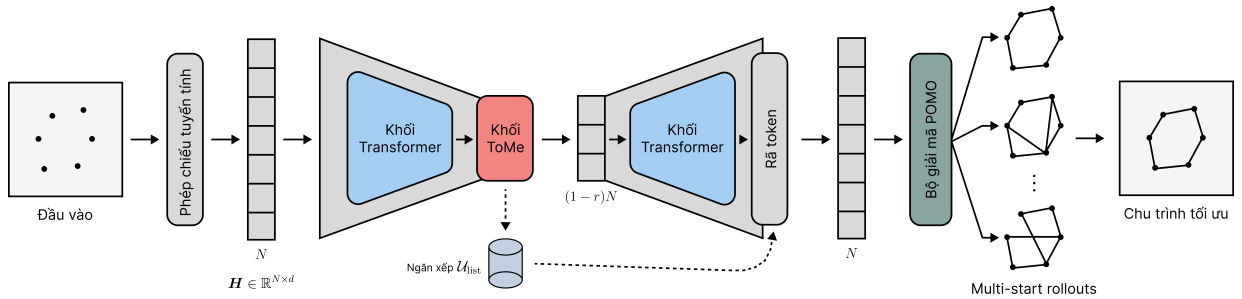
Phần III

Phương pháp đề xuất

Chương 13

Tổng quan kiến trúc

Hình 13.1 minh họa quy trình hoạt động của kiến trúc ToMe-POMO đề xuất. Quá trình xử lý được chia thành ba giai đoạn chính: mã hóa đồ thị, nén bằng thuật toán ToMe và giải mã đa điểm khởi đầu. Đầu tiên, một đồ thị TSP đầu vào gồm N đỉnh với tọa độ trong không gian Euclid được đưa qua phép chiếu tuyến tính (linear projection) để ánh xạ thành ma trận đặc trưng ban đầu có kích thước $\mathbb{R}^{N \times d}$. Ma trận này tuần tự đi qua các khối Transformer (màu xanh lam) để trích xuất ngữ nghĩa toàn cục. Điểm mới của kiến trúc nằm ở việc bố trí khối ToMe (màu đỏ) tại các lớp trung gian của bộ mã hóa, tạo ra một cấu trúc thắt cổ chai (bottleneck) về mặt không gian. Tại đây, số lượng token được chủ động cắt giảm, làm cho chiều dài của khối dữ liệu (biểu diễn bằng dãy các hình chữ nhật màu xám) ngắn lại đáng kể. Sự cắt giảm này giúp các khối Transformer ở tầng sâu hơn hoạt động với độ phức tạp tính toán và bộ nhớ được tối ưu hóa. Để đảm bảo tính tương thích với bộ giải mã theo thuật toán POMO gốc, mọi thao tác nén tại khối ToMe đều sinh ra một hàm lưu vết (được đẩy vào ngăn xếp hình trụ $\mathcal{U}_{\text{list}}$ bên dưới). Ngay trước khi kết thúc pha mã hóa, hệ thống thực hiện reverse token. Module này lấy các hàm khôi phục từ ngăn xếp để trả không gian đặc trưng về lại kích thước nguyên bản N đỉnh. Cuối cùng, bộ giải mã POMO (màu xanh rêu) nhận đầu vào là đồ thị đã được mã hóa đầy đủ, tiến hành giải mã đa điểm khởi đầu và xuất ra lộ trình Hamilton tối ưu.



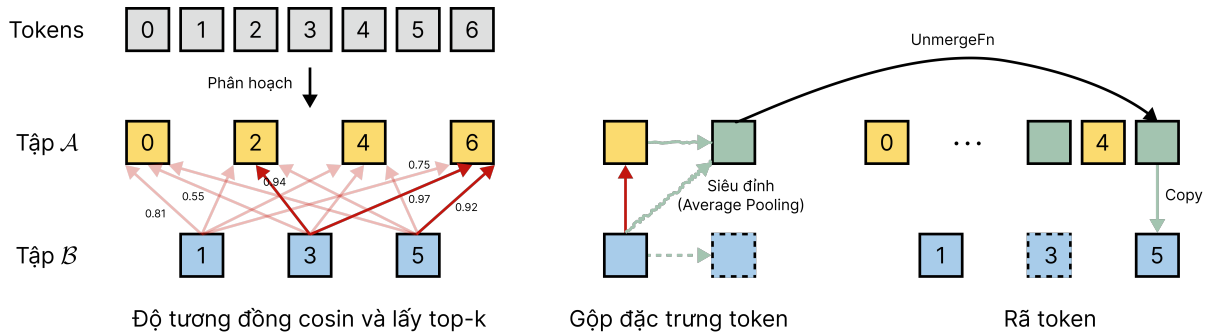
Hình 13.1. Tổng quan kiến trúc mạng ToMe-POMO. Hệ thống tích hợp một nút thắt cổ chai không gian vào bộ mã hóa thông qua ToMe, giúp giảm thiểu độ phức tạp của các lớp Transformer sâu. Ngăn xếp hình trụ lưu trữ các hàm đóng gói để phục hồi đồ thị về nguyên trạng trước khi đưa vào bộ giải mã POMO.

Nguồn: Tác giả.

Chương 14

Tích hợp ToMe vào luồng đồ thị

Hình 14.1 mô tả chi tiết thuật toán diễn ra bên trong khối ToMe và module rã token. Thay vì sử dụng các thuật toán gom cụm (clustering) tốn kém, hệ thống áp dụng kỹ thuật ghép cặp đồ thị hai phía (bipartite matching) [Bol+23]. Quá trình bắt đầu bằng việc phân hoạch N token đầu vào (dãy ô màu xám trên cùng) thành hai tập hợp rời rạc dựa trên chỉ số vị trí: tập đích A chứa các token chẵn (các ô màu vàng 0, 2, 4, 6) và tập nguồn B chứa các token lẻ (các ô màu xanh dương 1, 3, 5). Mũi tên đỏ đại diện cho quá trình trích xuất top- k cạnh ghép cặp dựa trên tính toán ma trận tương đồng cosine, gộp từ tập nguồn sang tập đích. Các token lẻ không có sự tương đồng cosine mạnh mẽ (như token số 1) sẽ bị bỏ qua và giữ nguyên trạng thái. Ở bước gộp đặc trưng, khi một token lẻ được gộp vào một token chẵn, chúng tạo thành một siêu đỉnh mới (ô màu xanh lá) chứa đặc trưng tổng hợp của cả hai thông qua phép toán trung bình cộng (average pooling). Đặc biệt, hệ thống để lại một không gian ảo (biểu diễn bằng ô vuông nét đứt) để đóng vai trò làm con trỏ lưu vết bộ nhớ. Cuối cùng, ở bước rã token, hệ thống sử dụng các con trỏ lưu vết này để thực thi phép phân tán (scatter). Đặc trưng của siêu đỉnh (ô màu xanh lá) được sao chép và trả ngược trở lại các ô nét đứt. Cơ chế này đảm bảo mọi token, dù bị ẩn đi trong quá trình tính toán trung gian, vẫn nhận được đầy đủ sự cập nhật đặc trưng khi kết thúc quá trình mã hóa.



Hình 14.1. Cơ chế nén và rã token hai phía. Các token được phân hoạch chẵn lẻ để thiết lập đồ thị hai phía. Thuật toán tìm kiếm các cạnh có độ tương đồng cao nhất (mũi tên đỏ) để gộp thành siêu đỉnh (màu xanh lá), đồng thời để lưu vết (nét đứt) nhằm phục vụ quá trình sao chép khôi phục ở cuối bộ mã hóa. *Nguồn: Tác giả.*

Chương 15

Thuật toán

15.1 Thuật toán nén đồ thị bằng gộp token hai phía

15.1.1 Trình bày thuật toán

Algorithm 1 Nén đồ thị bằng gộp token hai phía

```
1: procedure BIPARTITE_TOKEN_MERGING( $\mathbf{X}, \mathbf{s}, r$ )
2:    $k \leftarrow \lfloor N \times r \rfloor$ 
3:   if  $k = 0$  then return  $\mathbf{X}, \mathbf{s}$ , IdentityFunction
4:   end if
5:
6:   /* — Pha 1: Phân đôi và thiết lập đồ thị — */
7:    $\mathcal{I}_A \leftarrow \{2i\}_{i=0}^{\lfloor N/2 \rfloor}, \quad \mathcal{I}_B \leftarrow \{2i+1\}_{i=0}^{\lfloor N/2 \rfloor}$ 
8:    $\mathbf{A} \leftarrow \mathbf{X}[\mathcal{I}_A], \quad \mathbf{B} \leftarrow \mathbf{X}[\mathcal{I}_B]$ 
9:    $\tilde{\mathbf{A}} \leftarrow \mathbf{A} / \|\mathbf{A}\|_2, \quad \tilde{\mathbf{B}} \leftarrow \mathbf{B} / \|\mathbf{B}\|_2$ 
10:   $\mathbf{C} \leftarrow \tilde{\mathbf{B}} \cdot \tilde{\mathbf{A}}^\top$ 
11:
12:  /* — Pha 2: Cắt tỉa top- $k$  — */
13:   $\mathbf{m}_b \leftarrow \arg \max_a \mathbf{C}_{b,a} \quad \forall b \in \mathcal{I}_B$ 
14:   $\mathcal{E} \leftarrow \text{top-}k(\mathbf{m}, k)$ 
15:
16:  /* — Pha 3: Hàm phục hồi (closure) — */
17:  define UnmergeFn( $\mathbf{H}_m$ ):
18:    Khởi tạo:  $\mathbf{X}_{\text{unm}} \leftarrow \mathbf{0}^{N \times d}$ 
19:     $\mathbf{X}_{\text{unm}}[\mathcal{I}_A] \leftarrow \mathbf{H}_m$ 
20:     $\mathbf{X}_{\text{unm}}[\mathcal{I}_B] \leftarrow \mathbf{B}$ 
21:    for  $(b, a) \in \mathcal{E}$ :  $\mathbf{X}_{\text{unm}}[b] \leftarrow \mathbf{H}_m[a]$ 
22:    return  $\mathbf{X}_{\text{unm}}$ 
23:
24:  /* — Pha 4: Gộp đặc trưng và cập nhật trọng số — */
25:  Khởi tạo mảng gộp:  $\mathbf{X}_m \leftarrow \mathbf{A}, \quad \mathbf{s}_m \leftarrow \mathbf{s}_A$ 
26:  for mỗi cạnh  $(b, a) \in \mathcal{E}$  do
27:     $\mathbf{X}_m[a] \leftarrow \mathbf{X}_m[a] + \mathbf{B}[b]$ 
28:     $\mathbf{s}_m[a] \leftarrow \mathbf{s}_m[a] + \mathbf{s}_B[b]$ 
29:  end for
30:   $\mathbf{X}_{\text{rem}} \leftarrow \mathbf{X}_m / \mathbf{s}_m$ 
31:  return  $\mathbf{X}_{\text{rem}}, \mathbf{s}_m$ , UnmergeFn
32: end procedure
```

▷ Tập chỉ số chẵn và lẻ
▷ Trích xuất tập đích và tập nguồn
▷ Chuẩn hóa L_2
▷ Ma trận tương đồng cosine
▷ Trích xuất k cạnh ghép cặp ưu tiên
▷ Gán siêu đặc trưng vào vị trí chẵn
▷ Trả token không bị gộp về vị trí lẻ
▷ Sao chép đặc trưng siêu đỉnh
▷ Gộp đặc trưng token
▷ Cộng dồn trọng số token
▷ Lấy trung bình cộng (average pooling)
▷ Xuất ma trận nén, trọng số và hàm rã

15.1.2 Giải thích các biến số

Tham số đầu vào:

1. $\mathbf{X} \in \mathbb{R}^{N \times d}$: Ma trận đặc trưng đỉnh đầu vào với N đỉnh và d chiều ($d = 128$).
2. $\mathbf{s} \in \mathbb{R}^N$: Vector trọng số của từng đỉnh. Ban đầu được khởi tạo là vector toàn 1.
3. $r \in (0, 1)$: Tỷ lệ nén/gộp, quy định phần trăm số đỉnh sẽ bị triệt tiêu.

Biến trung gian:

1. k : Số lượng các đỉnh sẽ bị gộp ($k = \lfloor N \times r \rfloor$).
2. $\mathcal{I}_A, \mathcal{I}_B$: Tập hợp chứa các chỉ số nguyên (index). $\mathcal{I}_A$ chứa các số chẵn, $\mathcal{I}_B$ chứa các số lẻ.
3. $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{\frac{N}{2} \times d}$: Các ma trận con trích xuất từ $\mathbf{X}$, đại diện cho tập đích và tập nguồn.
4. $\mathbf{C} \in \mathbb{R}^{|\mathcal{B}| \times |\mathcal{A}|}$: Ma trận tương đồng cosine, đo lường khoảng cách ngữ nghĩa giữa tập đích và tập nguồn.
5. $\mathcal{E}$: Tập hợp các cạnh ghép cặp được chọn (chứa k cặp tuple (b, a)).

Tham số đầu ra:

1. $\mathbf{X}_{\text{rem}} \in \mathbb{R}^{(N-k) \times d}$: Ma trận đặc trưng đã được thu gọn sau khi gộp.
2. $\mathbf{s}_m$: Vector trọng số mới được cập nhật.
3. UnmergeFn: Hàm bao đóng (closure) chứa logic để khôi phục đồ thị.

15.1.3 Luồng thực thi

Pha 1: Phân hoạch và tính toán tương đồng. Mục đích chia đồ thị thành hai nửa để ghép cặp đồ thị hai phía, đồng thời đánh giá mức độ giống nhau giữa các đỉnh qua việc tính toán độ tương đồng cosine. Thuật toán chia tập đỉnh N thành hai nửa $\mathbf{A}$ (chỉ số chẵn) và $\mathbf{B}$ (chỉ số lẻ). Sau đó, các vector trong $\mathbf{A}$ và $\mathbf{B}$ được chuẩn hóa L_2 (chia cho độ lớn $\|\cdot\|_2$). Thao tác này khử đi magnitude của vector, chỉ giữ lại hướng. Phép nhân ma trận $\tilde{\mathbf{B}} \cdot \tilde{\mathbf{A}}^\top$ tính ra ma trận $\mathbf{C}$. Mỗi phần tử $C_{b,a}$ là điểm tương đồng cosine (từ -1 đến 1), phản ánh việc đỉnh b và đỉnh a đóng vai trò gần nhau như thế nào trong bài toán TSP hiện tại.

Pha 2: Lọc top- k . Mục đích xem những đỉnh nào thuộc tập nguồn $\mathbf{B}$ sẽ bị gộp đi. Với mỗi đỉnh $b \in \mathcal{I}_B$, thuật toán tìm ra đỉnh phù hợp nhất với nó trong tập $\mathbf{A}$ bằng hàm $\arg \max_a C_{b,a}$ và lưu vào mảng m_b . Tuy nhiên, không phải mọi đỉnh b đều bị gộp. Hàm top- k sẽ sắp xếp các cặp này theo độ tương đồng giảm dần, và chỉ gộp đúng k cặp có điểm cosine cao nhất. Tập $\mathcal{E}$ lúc này chính là danh sách k cặp đỉnh được gộp.

Pha 3: Xây dựng cơ chế phục hồi token. Mục đích chuẩn bị sẵn cơ chế để bộ giải mã POMO sau này có thể nhận lại đủ N đỉnh. Thuật toán định nghĩa (nhưng chưa chạy) một hàm bao đóng UnmergeFn. Hàm này nhận vào ma trận đã bị nén ở tương lai ($\mathbf{H}_m$). Khi được gọi, nó tạo một không gian rỗng $\mathbf{X}_{\text{unm}}$ đủ N chỗ. Nó cho các siêu đỉnh $\mathbf{H}_m$ về lại các vị trí chẵn và cho các đỉnh lẻ (không bị gộp) về lại vị trí lẻ. Những đỉnh lẻ b nào đã bị gộp ở tập $\mathcal{E}$, nay sẽ được phục hồi đặc trưng bằng cách sao chép y nguyên đặc trưng của siêu đỉnh a tương ứng ($\mathbf{H}_m[a]$), bảo toàn đặc trưng các token.

Pha 4: Gộp đặc trưng. Mục đích thực hiện nén đồ thị. Khởi tạo ma trận siêu đỉnh $\mathbf{X}_m$ bằng đúng tập đích $\mathbf{A}$. Duyệt qua danh sách $\mathcal{E}$, đỉnh b nào bị gộp sẽ được đem cộng dồn vector đặc trưng của nó vào vector của đỉnh a . Đồng thời, trọng số $\mathbf{s}_m[a]$ cũng được cộng thêm 1. Sau đó dùng average pooling. Hàm trả về ma trận $\mathbf{X}_{\text{rem}}$ (nhỏ gọn hơn), vector trọng số $\mathbf{s}_m$, và hàm lưu vết UnmergeFn.

15.2 Bộ mã hóa trong kiến trúc ToMe-POMO đề xuất

15.2.1 Trình bày thuật toán

Algorithm 2 Bộ mã hóa kiến trúc ToMe-POMO

```

1: procedure TOMEPOMOENCODER( $\mathbf{X}_{\text{raw}}$ , tỷ lệ nén  $r$ , tập các lớp cần nén  $\mathcal{K}_{\text{tome}}, \theta$ )
2:    $\mathbf{H} \leftarrow \text{InitEmbedding}(\mathbf{X}_{\text{raw}})$  ▷ Nhúng tọa độ, không gian  $\mathbb{R}^{N \times d}$ 
3:    $\mathbf{s} \leftarrow \mathbf{1}^{N \times 1}$  ▷ Khởi tạo trọng số token ban đầu
4:    $\mathcal{U}_{\text{list}} \leftarrow \emptyset$  ▷ Danh sách ngăn xếp lưu các hàm rã token
5:
6:   /* — Pha 1: Tính toán attention và nén token — */
7:   for  $\ell = 0$  to  $L - 1$  do ▷ Duyệt qua  $L$  lớp Transformer của hệ thống
8:      $\mathbf{H} \leftarrow \text{MultiHeadAttention}_{\ell}(\mathbf{H})$  ▷ Học ngữ nghĩa đồ thị
9:     if  $\ell \in \mathcal{K}_{\text{tome}}$  then
10:       /* Cơ chế nén (thuật toán 1) */
11:        $\mathbf{H}, \mathbf{s}, \text{UnmergeFn} \leftarrow \text{BipartiteTokenMerging}(\mathbf{H}, \mathbf{s}, r)$ 
12:        $\mathcal{U}_{\text{list}}.\text{push}(\text{UnmergeFn})$  ▷ Lưu vết vào ngăn xếp
13:     end if
14:   end for
15:
16:   /* — Pha 2: Phục hồi không gian (rã token) — */
17:   while  $\mathcal{U}_{\text{list}}$  is not empty do
18:      $\text{UnmergeFn} \leftarrow \mathcal{U}_{\text{list}}.\text{pop}()$  ▷ Lấy hàm khôi phục theo thứ tự LIFO
19:     /* Cơ chế rã (pha 3 thuật toán 1) */
20:      $\mathbf{H} \leftarrow \text{UnmergeFn}(\mathbf{H})$  ▷ Phục hồi kích thước về  $N \times d$ 
21:   end while
22:   return  $\mathbf{H}$  ▷ Xuất ma trận nhúng cho bộ giải mã POMO
23: end procedure

```

15.2.2 Giải thích các biến số

Tham số đầu vào:

1. $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{N \times 2}$: Ma trận chứa tọa độ không gian 2D ban đầu của N đỉnh trong bài toán TSP.
2. $r \in (0, 1)$: Tỷ lệ nén/gộp, quy định phần trăm số đỉnh sẽ bị triệt tiêu, trong thuật toán 1.
3. $\mathcal{K}_{\text{tome}}$: Tập hợp chứa chỉ số của các lớp Transformer (ví dụ: $\{2\}$ hoặc $\{2, 3\}$) được chỉ định để áp dụng cơ chế nén đồ thị.
4. θ : Tập hợp các tham số (parameters) có thể tinh chỉnh của bộ mã hóa.

Biến trung gian:

1. $\mathbf{H}$: Ma trận biểu diễn ẩn. Kích thước của $\mathbf{H}$ thay đổi động trong quá trình thực thi: bắt đầu ở $\mathbb{R}^{N \times d}$, giảm xuống $\mathbb{R}^{(1-r)N \times d}$ sau khi đi qua các lớp nén và phục hồi về $\mathbb{R}^{N \times d}$ ở cuối thuật toán ($d = 128$).
2. $\mathbf{s} \in \mathbb{R}^{N \times 1}$: Vector lưu trữ kích thước cộng dồn của các token, phục vụ cho phép toán lấy average pooling trong thuật toán 1.
3. $\mathcal{U}_{\text{list}}$: Cấu trúc dữ liệu ngăn xếp, dùng để lưu trữ các hàm bao đóng (closure) chứa quy tắc rã token. Ngăn xếp hoạt động theo nguyên lý LIFO (Last-In-First-Out).

15.2.3 Luồng thực thi

Trước hết, chuyển đổi dữ liệu thô thành định dạng tensor phù hợp cho mạng neural và cấp phát bộ nhớ. Thuật toán sử dụng một phép chiếu tuyến tính để nhúng tọa độ 2D của các đỉnh lên không gian đặc trưng d -chiều, tạo ra ma trận $\mathbf{H}$. Vector $\mathbf{s}$ được khởi tạo với các giá trị 1, biểu thị mỗi token ban đầu là một đỉnh đơn lẻ. Ngăn xếp $\mathcal{U}_{\text{list}}$ được khởi tạo rỗng.

Pha 1: Tính toán attention và nén token. Mục tiêu trích xuất đặc trưng toàn cục của đồ thị thông qua các khối Transformer, đồng thời giảm dần chiều không gian tại các lớp được chỉ định để tiết kiệm chi phí tính toán. Đồ thị được truyền qua L lớp Transformer. Tại mỗi lớp ℓ , cập nhật ma trận đặc trưng $\mathbf{H}$ qua cơ chế multi-head attention bằng cách thu thập thông tin ngữ nghĩa từ các đỉnh khác trong đồ thị. Sau khi tính toán attention, hệ thống kiểm tra điều kiện $\ell \in \mathcal{K}_{\text{tome}}$. Nếu lớp hiện tại được cấu hình để nén, hàm BipartiteTokenMerging (thuật toán 1) sẽ được gọi. Thao tác nén cập nhật ma trận $\mathbf{H}$ về một kích thước nhỏ hơn, đồng thời cập nhật trọng số $\mathbf{s}$. Quan trọng nhất, hàm này trả về một đối tượng hàm (closure) UnmergeFn mang thông tin về các cạnh đã bị gộp. Hàm UnmergeFn lập tức được đẩy vào ngăn xếp $\mathcal{U}_{\text{list}}$. Nhờ thiết kế này, dữ liệu đi qua các lớp Transformer tiếp theo với kích thước đã được thu gọn, giúp giảm trực tiếp độ phức tạp bậc hai $\mathcal{O}(N^2)$ của cơ chế self-attention.

Pha 2: Rã token. Mục đích tái cấu trúc ma trận đặc trưng về lại kích thước ban đầu ($N \times d$) để đảm bảo tính tương thích với cấu trúc đầu vào của bộ giải mã POMO. Sau khi hoàn tất quá trình trích xuất đặc trưng ở toàn bộ L lớp, thuật toán tiến hành xử lý các hàm khôi phục đang nằm trong ngăn xếp $\mathcal{U}_{\text{list}}$. Vòng lặp while rút lần lượt các hàm UnmergeFn ra khỏi ngăn xếp. Tính chất LIFO của ngăn xếp đảm bảo rằng đồ thị được rã ra theo đúng trình tự ngược lại với lúc bị nén (hàm nào gộp sau cùng sẽ được rã đầu tiên). Khi thực thi hàm UnmergeFn($\mathbf{H}$), ma trận đặc trưng được nội suy: các token gộp được khôi phục vị trí và nhận các giá trị đặc trưng được sao chép từ siêu đỉnh đại diện. Sau khi ngăn xếp rỗng, ma trận $\mathbf{H}$ lấy lại kích thước đầy đủ $N \times d$ và được trả về làm kết quả đầu ra, kết thúc bộ mã hóa.

15.3 Huấn luyện và đánh giá Tome-POMO

15.3.1 Trình bày thuật toán

Algorithm 3 Huấn luyện và đánh giá ToMe-POMO

```

1: procedure MAIN( $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}, N, r, \mathcal{K}_{\text{tome}}, E$ )
2:   Khởi tạo mô hình ToMe-POMO với trọng số  $\theta$  ▷ Gồm bộ mã hóa nén và bộ giải mã
3:   Khởi tạo bộ tối ưu hóa Adam có learning rate  $\eta$ 
4:   Khởi tạo biến lưu trọng số tốt nhất  $\theta^* \leftarrow \theta$ 
5:
6:   /* — Pha 1: Huấn luyện — */
7:   for epoch  $e = 1$  to  $E$  do
8:     for mỗi mini-batch  $\mathcal{B} \subset \mathcal{D}_{\text{train}}$  do
9:       for mỗi đồ thị  $\mathbf{X} \in \mathcal{B}$  do
10:        /* Truyền lan tiến qua Bộ mã hóa nén (Thuật toán 3) */
11:         $\mathbf{H}_{\text{node}} \leftarrow \text{ToMeEncoder}(\mathbf{X}, r, \mathcal{K}_{\text{tome}}, \theta)$ 
12:        /* Giải mã đa khởi điểm POMO (Multi-start Rollouts) */
13:         $\{\pi_1, \dots, \pi_N\} \leftarrow \text{Decoder}(\mathbf{H}_{\text{node}}, \theta)$  ▷ Sinh  $N$  chu trình
14:        /* Tính toán hàm mục tiêu REINFORCE */
15:         $c_j \leftarrow \text{Cost}(\pi_j) \quad \forall j \in \{1, \dots, N\}$  ▷ Độ dài vô hướng  $c_j$ 
16:         $b_{\mathbf{X}} \leftarrow \frac{1}{N} \sum_{j=1}^N c_j$  ▷ Đường cơ sở
17:      end for
18:      /* Tính Gradient trung bình trên toàn mini-batch */
19:       $\nabla_{\theta} J(\theta) \leftarrow \frac{1}{|\mathcal{B}|N} \sum_{\mathbf{X} \in \mathcal{B}} \sum_{j=1}^N (c_j - b_{\mathbf{X}}) \nabla_{\theta} \log p_{\theta}(\pi_j)$ 
20:      /* Cập nhật trọng số */
21:       $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$ 
22:    end for
23:    Cập nhật  $\theta^*$  nếu hiệu suất validation tại epoch  $e$  đạt cực đại
24:  end for
25:
26:  /* — Pha 2: Đánh giá — */
27:  Khôi phục trọng số tốt nhất:  $\theta \leftarrow \theta^*$ 
28:  Tắt cơ chế Gradient và chuyển mô hình sang chế độ eval()
29:   $\mathbf{H}_{\text{test}} \leftarrow \text{ToMeEncoder}(\mathcal{D}_{\text{test}}, r, \mathcal{K}_{\text{tome}}, \theta^*)$ 
30:   $\{\pi_1, \dots, \pi_N\} \leftarrow \text{Decoder}(\mathbf{H}_{\text{test}}, \theta^*)$ 
31:   $\mathbf{c}_{\text{test}}[j] \leftarrow \text{Cost}(\pi_j) \quad \forall j \in \{1, \dots, N\}$ 
32:   $\mathbf{c}_{\text{best}} \leftarrow \min(\mathbf{c}_{\text{test}})$  ▷ Chọn chu trình ngắn nhất làm kết quả
33:  return  $\theta^*, \text{mean}(\mathbf{c}_{\text{best}})$  ▷ Xuất mô hình tối ưu và chi phí trung bình
34: end procedure

```

15.3.2 Giải thích các biến số

Tham số cấu hình:

1. $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}$: Tập dữ liệu huấn luyện và kiểm thử.
2. $\mathcal{B} \subset \mathcal{D}_{\text{train}}$: mini-batch chứa các đồ thị độc lập được trích xuất ở mỗi bước lặp.
3. $N, r, \mathcal{K}_{\text{tome}}$: Lần lượt là kích thước đồ thị, tỷ lệ nén, và tập chỉ số các lớp mã hóa áp dụng nén.
4. E : Số lượng chu kỳ huấn luyện lớn nhất (max epochs).
5. η : Tốc độ học của thuật toán tối ưu hóa Adam.

Tham số mô hình:

1. θ : Biến không gian trạng thái, đại diện cho toàn bộ trọng số của mạng neural (bao gồm cả bộ mã hóa và bộ giải mã).
2. θ^* : Trọng số tối ưu nhất được sao lưu trên tập kiểm định (validation set) trong quá trình huấn luyện.

Biến tối ưu hóa:

1. π_j : Vector thứ tự đỉnh, biểu diễn chu trình Hamilton xuất phát từ đỉnh thứ j .
2. c_j : Giá trị vô hướng đại diện cho chi phí (tổng chiều dài vật lý) của chu trình π_j .
3. b_X : Đường cơ sở, tính bằng giá trị trung bình chi phí của N chu trình trên cùng một đồ thị X .
4. $\nabla_{\theta} J(\theta)$: Vector gradient của hàm mục tiêu REINFORCE, định hướng cập nhật trọng số để cực tiểu hóa chiều dài chu trình.

15.3.3 Luồng thực thi

Pha 1: Vòng lặp huấn luyện REINFORCE. Mục tiêu tối ưu hóa trọng số θ của mạng neural thông qua phương pháp policy gradient, kết hợp cơ chế đường cơ sở của kiến trúc POMO để giảm thiểu phương sai khi tính đạo hàm. Thuật toán duyệt qua E epoch, mỗi epoch xử lý tuần tự các mini-batch $\mathcal{B}$. Đối với mỗi đồ thị X , bộ mã hóa ToMe tiến hành nén và mã hóa không gian, trả về ma trận đặc trưng đỉnh H_{node} nguyên vẹn kích thước. Bộ giải mã nhận H_{node} và khai triển cơ chế đa điểm khởi đầu. Nó buộc hành trình lần lượt bắt đầu từ N đỉnh khác nhau, sinh ra N chu trình $\pi_1, \dots, \pi_N$ độc lập. Hàm Cost tính toán tổng chiều dài vô hướng c_j của từng chu trình. Giá trị trung bình của N chiều dài này được sử dụng làm đường cơ sở b_X . Đây là đặc trưng cốt lõi của POMO: sử dụng chính kết quả của mô hình làm baseline thay vì phải huấn luyện một mạng critic phụ trợ như kiến trúc actor-critic [Kwo+20]. Gradient $\nabla_{\theta} J(\theta)$ được tính toán dựa trên hệ số ưu thế (advantage) là $(c_j - b_X)$. Nếu một chu trình j có chiều dài ngắn hơn mức trung bình ($c_j - b_X < 0$), xác suất chọn hành động của chu trình đó sẽ được tăng cường. Quá trình này được trung bình hóa trên toàn bộ đồ thị trong mini-batch $\mathcal{B}$ để đảm bảo độ ổn định của gradient trước khi thực hiện bước cập nhật Adam.

Pha 2: Đánh giá và suy luận. Mục tiêu kiểm tra hiệu năng thực tế của mô hình trên dữ liệu chưa từng thấy, áp dụng chiến lược chọn lọc kết quả tốt nhất. Trọng số mạng được khôi phục về trạng thái tối ưu nhất θ^* . Mạng được chuyển sang chế độ suy luận (eval()). Tập dữ liệu kiểm thử $\mathcal{D}_{\text{test}}$ được truyền qua mạng. Tương tự như pha huấn luyện, mô hình sinh ra N chu trình song song cho mỗi đồ thị. Ở giai đoạn này, thuật toán áp dụng chiến lược tìm kiếm cực tiểu cục bộ: so sánh chiều dài của N chu trình tạo ra và chỉ trích xuất đúng chu trình có chi phí thấp nhất min(c_{test}). Chi phí trung bình của các chu trình ngắn nhất này được tính toán để làm cơ sở đối chiếu với các thuật toán khác.

Chương 16

Phân tích toán học

Contents

15.1 Thuật toán nén đồ thị bằng gộp token hai phía	30
15.1.1 Trình bày thuật toán	30
15.1.2 Giải thích các biến số	31
15.1.3 Luồng thực thi	31
15.2 Bộ mã hóa trong kiến trúc ToMe-POMO đề xuất	32
15.2.1 Trình bày thuật toán	32
15.2.2 Giải thích các biến số	32
15.2.3 Luồng thực thi	33
15.3 Huấn luyện và đánh giá Tome-POMO	34
15.3.1 Trình bày thuật toán	34
15.3.2 Giải thích các biến số	34
15.3.3 Luồng thực thi	35

Bài toán cần tối ưu trên một đồ thị gồm N đỉnh. Khi đó đầu vào của mạng neural là một tensor biểu diễn $\mathcal{X} \in T_{B \times N \times 2}$, trong đó B là kích thước một batch. Mô hình sử dụng kiểu dữ liệu cho giá trị các tham số là `Float32`, khi đó mỗi giá trị vô hướng chiếm 4 bytes bộ nhớ vật lý trong `VRAMs` của `GPU`.

Định lý 1 (Bộ nhớ kích hoạt của bộ mã hóa – Encoder Activation Memory [Kor+23]). Cho trước độ dài chuỗi s , kích thước batch b , số chiều ẩn h và số attention head độc lập a . Khi đó bộ nhớ kích hoạt cho mỗi lớp Transformer trong quá trình lan truyền ngược đúng bằng

$$M_{act} = sbh \left(34 + 5 \frac{as}{h} \right) \quad (\text{bytes}).$$

Định lý 2 (Độ phức tạp tính toán của nhân ma trận tổng quát (GEMM)). Cho hai tensor $\mathcal{A} \in T_{B \times M \times K}(\mathbb{R})$ và $\mathcal{B} \in T_{B \times K \times N}(\mathbb{R})$. Phép nhân ma trận theo batch B được tính toán bởi $\mathcal{C} = \text{BMM}(\mathcal{A}, \mathcal{B})$, thực hiện trên từng thành phần batch $\mathcal{C}_i = \mathcal{A}_i \mathcal{B}_i$ với mọi $i = 1, \dots, B$. Khi đó số lượng phép tính Floating-point Operations Per Second (FLOPs) cần thiết bằng $2BMNK$.

16.1 Kiến trúc POMO cơ sở

Kiến trúc POMO cơ sở dựa trên một mạng xương sống AM, được chia ra thành kiến trúc mã hóa Transformer đa lớp và giải mã Transformer tự hồi quy.

Bộ mã hóa bao gồm $L = 6$ lớp, mỗi lớp sử dụng cơ chế multi-head self-attention nhằm chiếu đầu vào qua một không gian nhúng ẩn với số chiều $d = 128$, sử dụng $a = 8$ attention head độc lập nhau. Sau đó một mạng truyền thẳng mở rộng không gian nhúng ẩn đến số chiều trung gian $d_{\text{ff}} = 512$ trước khi trả về lại $d = 128$.

Bộ giải mã khai thác tính đối xứng của **POMO**, bằng cách đánh giá N quỹ đạo song song cho mỗi đồ thị riêng lẻ trong mỗi batch. Ở mỗi bước giải mã tự hồi quy, các truy vấn nút ngữ cảnh sẽ xử lý các đầu ra chưa bị che khuất (masked) của bộ mã hóa để tính toán xác suất hành động cho việc lựa chọn nút tiếp theo.

16.1.1 Độ phức tạp của bộ mã hóa

Định lý 3 (Độ phức tạp của bộ mã hóa kiến trúc **POMO**). *Độ phức tạp không gian và tính toán của bộ mã hóa trong kiến trúc **POMO** được xác định bởi:*

1. Độ phức tạp bộ nhớ: $M_{enc}^{POMO}(N) = \mathcal{O}(5LBaN^2 + 34LBdN)$.
2. Độ phức tạp tính toán: $C_{enc}^{POMO}(N) = \mathcal{O}(4LBdN^2 + 24LBd^2N)$.

Chứng minh. Áp dụng định lý 1 vào mô hình **POMO**, thay $s = N, b = B, h = d, a = a$, ngoài ra chú ý rằng có L lớp nên độ phức tạp không gian cho bộ mã hóa sẽ bằng

$$M_{enc}^{POMO}(N) = L \cdot N \cdot B \cdot d \cdot \left(34 + 5 \cdot \frac{a \cdot N}{d} \right) = 34LBdN + 5LBaN^2,$$

hay nói cách khác độ phức tạp bộ nhớ sẽ là $\mathcal{O}(5LBaN^2 + 34LBdN)$.

Về mặt tính toán, số lượng phép tính **FLOPs** được tính toán chặt chẽ dựa trên các mô hình tính toán phân cứng đã được thiết lập bởi Narayanan và cộng sự (2021) cùng với Korthikanti và cộng sự (2023) [Nar+21; Kor+23]. Đối với lượt truyền thông tin xuôi của một tầng Transformer đơn lẻ, có các phép **GEMM**:

1. Các phép chiếu tuyến tính (Q, K, V): Việc chuyển đổi đầu vào thành các ma trận Query, Key và Value đòi hỏi 3 phép **GEMM** riêng biệt với kích thước $(B \times N \times d) \times (d \times d)$, đòi hỏi $3 \times (2BNd^2) = 6Bd^2N$ phép tính **FLOPs**.
2. Tính toán ma trận attention ($QK^\top$): Phép nhân tích vô hướng giữa Q và $K^\top$, đòi hỏi $2BdN^2$ phép tính **FLOPs**.
3. Tính toán attention trên các giá trị (AV): Nhân các xác suất attention A với V , đòi hỏi $2BdN^2$ phép tính **FLOPs**.
4. Phép chiếu tuyến tính sau attention: Việc chiếu các đầu (heads) đã được nối lại về chiều d ban đầu, đòi hỏi $2Bd^2N$ phép tính **FLOPs**.
5. Mạng neural truyền thẳng: Lớp MLP hai tầng mở rộng chiều ẩn từ d lên $4d$ ($128 \rightarrow 512$) và sau đó chiếu ngược lại từ $4d$ về d , đòi hỏi $(2BNd \cdot 4d) + (2BN4d \cdot d) = 16Bd^2N$ phép tính **FLOPs**.

Tổng hợp các kết quả trên, chú ý nhân với L lớp, khi đó ta có độ phức tạp tính toán sẽ là $C_{enc}^{POMO}(N) = \mathcal{O}(4LBdN^2 + 24LBd^2N)$. $\square$

16.1.2 Độ phức tạp bộ giải mã

Định lý 4 (Độ phức tạp của bộ giải mã kiến trúc **POMO**). *Dưới tác động của cơ chế triển khai song song (parallel rollout) và causal triangular masking, độ phức tạp về bộ nhớ và tính toán của bộ giải mã trong kiến trúc **POMO** cơ bản sẽ tăng tiến theo đa thức bậc ba:*

1. Độ phức tạp bộ nhớ: $M_{dec}^{POMO}(N) = \mathcal{O}(6BaN^3 + 6BaN^2)$.
2. Độ phức tạp tính toán: $C_{dec}^{POMO}(N) = \mathcal{O}(4BdN^3 + 24Bd^2N^2)$.

Chứng minh. Kiến trúc **POMO** yêu cầu bộ giải mã phải xử lý đồng thời N quỹ đạo song song. Tại mỗi bước giải mã tự hồi quy $t \in [1, N]$, ma trận cross-attention có kích thước là $B \times a \times N_{rollouts} \times N_{đỉnh}$. Vì số lượng rollout và số lượng đỉnh đều bằng N , nên kích thước ma trận tại bước t sẽ là $B \times a \times N \times N$. Khi một đỉnh đã được chọn ở bước t , nó sẽ bị che khuất bằng cách đặt giá trị logit về $-\infty$. Nhờ cơ chế causal triangular

masking này, thư viện Autograd của PyTorch có thể bỏ qua việc tính toán gradient cho các vùng $-\infty$. Ở bước t , mỗi rollout sẽ có chính xác $t - 1$ node bị khóa. Tổng số các phần tử chưa bị khóa, cần cấp phát gradient qua N bước sẽ tạo thành một cấp số cộng:

$$B \cdot a \cdot N \sum_{t=1}^N (N - (t - 1)) = \frac{1}{2} BaN^3 + \frac{1}{2} BaN^2.$$

Với định dạng **Float32**, việc lưu trữ logits (trước softmax), xác suất (sau softmax) và gradient (lan truyền ngược) tiêu tốn 12 bytes cho mỗi phần tử chưa bị khóa, tính ra đúng bằng $6BaN^3 + 6BaN^2$ bytes. Như vậy độ phức tạp bộ nhớ sẽ là $M_{\text{dec}}^{\text{POMO}}(N) = \mathcal{O}(6BaN^3 + 6BaN^2)$.

Về mặt tính toán, bộ giải mã thực hiện quá trình sinh tự hồi quy qua tổng cộng N bước. Tại mỗi bước, nó xử lý song song N rollout. Do đó, tổng chiều dài chuỗi thực tế mà các lớp chiếu tuyến tính (linear projection – Q, post-attention và FFN) phải nhận qua tất cả các bước là $N \times N = N^2$, tương đương $24Bd^2N^2$ phép tính FLOPs. Trong cơ chế cross-attention ở bước t , các Query ($B \times N \times d$) sẽ xem xét Key và Value của bộ mã hóa ($B \times N \times d$). Mỗi thao tác $QK^\top$ và AV tiêu tốn $2BdN^2$ phép tính FLOPs cho mỗi bước. Tính gộp lại sau N bước, ta có $N \times (4BdN^2) = 4BdN^3$ phép tính FLOPs. Cộng tất cả lại, ta thu được độ phức tạp tính toán $C_{\text{dec}}^{\text{POMO}}(N) = \mathcal{O}(4BdN^3 + 24Bd^2N^2)$. $\square$

16.2 Kiến trúc ToMe-POMO đề xuất

16.2.1 Độ phức tạp của bộ mã hóa

Định lý 5 (Độ phức tạp của bộ mã hóa kiến trúc ToMe-POMO). *Với tỷ lệ gộp r được áp dụng tại lớp thứ k , độ phức tạp về bộ nhớ và tính toán của bộ mã hóa trong kiến trúc ToMe-POMO đề xuất được xác định bởi:*

1. *Độ phức tạp bộ nhớ:* $M_{\text{enc}}^{\text{ToMe-POMO}}(N, r, k) = \mathcal{O}(5Ba[k + (L - k)(1 - r)]N^2 + 34Bd[k + (L - k)(1 - r)]N)$.
2. *Độ phức tạp tính toán:* $C_{\text{enc}}^{\text{ToMe-POMO}}(N, r, k) = \mathcal{O}(4Bd[k + (L - k)(1 - r)]N^2 + 24Bd^2[k + (L - k)(1 - r)]N)$.

Chứng minh. Bộ mã hóa sẽ xử lý toàn bộ chuỗi N trong k lớp đầu tiên, sau đó xử lý chuỗi đã được nén $N_{\text{rem}} = (1 - r)N$ cho $(L - k)$ lớp còn lại. Trong công thức độ phức tạp bộ nhớ của định lý 3, lần lượt thay (N, L) bởi (N, k) và $(N_{\text{rem}}, L - k)$ ta được $M_{\text{enc}}^{\text{ToMe-POMO}}(N, r, k) = k(5BaN^2 + 34BdN) + (L - k)(5Ba((1 - r)N)^2 + 34Bd(1 - r)N)$. Tương tự, trong công thức độ phức tạp tính toán của định lý 3, cũng lần lượt thay (N, L) bởi (N, k) và $(N_{\text{rem}}, L - k)$ ta được $C_{\text{enc}}^{\text{ToMe-POMO}}(N, r, k) = k(4BdN^2 + 24Bd^2N) + (L - k)(4Bd((1 - r)N)^2 + 24Bd^2(1 - r)N)$.

Ngoài ra, ngay sau lớp thứ k , thuật toán thực thi nén đồ thị và lưu vết. Ở bước cuối cùng của bộ mã hóa, thuật toán lùi vết từ ngăn xếp để tiến hành rẽ. Chi phí phần cứng của hai toán tử này được tính toán như sau:

1. *Phép nén đồ thị:* Chi phí lớn nhất nằm ở phép **GEMM** để tính độ tương đồng cosine giữa tập đích và tập nguồn (mỗi tập có kích thước $\frac{N}{2}$). Kích thước phép nhân là $(B \times \frac{N}{2} \times d) \times (B \times d \times \frac{N}{2})$, tiêu tốn $2Bd(\frac{N}{2})^2 = 0.5BdN^2$ **FLOPs**.
2. *Phép rẽ đồ thị:* Thao tác này mang bản chất là dịch chuyển dữ liệu thay vì tính toán toán học. Việc cấp phát zero-tensor để phục hồi không gian $\mathbb{R}^{B \times N \times d}$ tiêu tốn giới hạn phần cứng chính xác là $4BdN$ bytes. Sau đó, thao tác gán đề (scatter) dữ liệu từ siêu đỉnh về lại các token ban đầu không yêu cầu bất kỳ phép tính nào (0 **FLOPs**), mà chỉ tiêu tốn $\mathcal{O}(BdN)$ về truy xuất và gán bộ nhớ.

Tuy nhiên độ phức tạp của hai thao tác này không đáng kể. Do đó, việc lược bỏ chúng không làm thay đổi cấu trúc đa thức. Gộp các hạng tử tương đồng của $M_{\text{enc}}^{\text{ToMe-POMO}}$ và $C_{\text{enc}}^{\text{ToMe-POMO}}$ theo bậc đa thức, ta thu được kết quả như đã phát biểu. $\square$

16.2.2 Độ phức tạp của bộ giải mã

Định lý 6 (Độ phức tạp của bộ giải mã kiến trúc ToMe-POMO). *Độ phức tạp về bộ nhớ và tính toán của bộ giải mã trong kiến trúc ToMe-POMO đề xuất được xác định bởi:*

1. Độ phức tạp bộ nhớ: $M_{dec}^{ToMe-POMO}(N, r) = \mathcal{O}(6BaN^3 + 6BaN^2)$.
2. Độ phức tạp tính toán: $C_{dec}^{ToMe-POMO}(N, r) = \mathcal{O}(4BdN^3 + 24Bd^2N^2)$.

Chứng minh. Mặc dù kiến trúc ToMe-POMO can thiệp sâu vào việc giảm thiểu không gian đặc trưng trong quá trình mã hóa, nhưng độ phức tạp tính toán và bộ nhớ của bộ giải mã vẫn được duy trì đồng nhất với kiến trúc POMO. Nguyên nhân là do việc mã hóa được thực hiện tại vị trí cuối cùng của bộ mã hóa, phức hồi toàn vẹn kích thước ma trận đặc trưng về lại không gian $\mathbb{R}^{B \times N \times d}$. $\square$

16.2.3 Tính đẳng cự và bảo toàn không gian

Mệnh đề 1. *Giả sử $X_i, X_j \in \mathbb{R}^2$ là tọa độ không gian gốc và $h_i, h_j \in \mathbb{R}^d$ là các vector biểu diễn khóa đã được chuẩn hóa L_2 tại lớp $k = 2$. Thuật toán ghép cặp hai phía mềm thực hiện gộp các token bằng cách tối đa hóa độ tương đồng cosine $S_{ij} = h_i^\top h_j$. Đối với các lớp $k \leq 2$, thao tác này tương đương về mặt toán học với việc tối thiểu hóa khoảng cách Euclid trong không gian thực $\|X_i - X_j\|_2$.*

Chứng minh. Đối với các vector đã được chuẩn hóa L_2 (tức $\|h\| = 1$), khoảng cách Euclid trong không gian ẩn được biểu diễn qua độ tương đồng cosin:

$$\|h_i - h_j\|_2^2 = \|h_i\|^2 + \|h_j\|^2 - 2(h_i^\top h_j) = 1 + 1 - 2S_{ij} = 2(1 - S_{ij}).$$

Từ đó ta có $D_{\text{latent}}(h_i, h_j)^2 = 2(1 - S_{ij})$. Do đó, việc tối đa hóa S_{ij} về mặt giải tích là hoàn toàn đồng nhất với việc tối thiểu hóa $D_{\text{latent}}(h_i, h_j)$.

Theo nghiên cứu của Khromov & Singh, giới hạn của hằng số Lipschitz (đại diện cho mức độ biến dạng của không gian đặc trưng) duy trì ở mức rất thấp và tiệm cận giá trị khởi tạo trong đúng 2 lớp đầu tiên của mạng, bảo toàn tính đẳng cự của không gian đầu vào hai chiều [KS24].

Vì cấu trúc khoảng cách được bảo toàn, ta thu được sự tương đương chính xác giữa việc chọn phần tử tương đồng nhất trong không gian ẩn và phần tử gần nhất trong không gian thực:

$$\arg \max_{X_j \in \mathbf{A}} S_{ij} \equiv \arg \min_{X_j \in \mathbf{A}} \|X_i - X_j\|_2.$$

Điều này chứng minh rằng tại lớp $k = 2$, việc ghép cặp dựa trên đặc trưng ẩn thực chất là một phép chiếu tìm láng giềng gần nhất trong không gian thực. $\square$

16.2.4 Phân tích sai số

Định lý 7 (Sai số nội suy cục bộ). *Gọi $\mathcal{X}_N$ là tập hợp gồm N biến ngẫu nhiên độc lập có phân phối đều trong một hình vuông đơn vị. Qua thuật toán lựa chọn gộp token nói trên, đúng rN token từ tập nguồn $\mathbf{B}$ (với $|\mathbf{B}| = 0.5N$) sẽ được gộp với các láng giềng gần nhất của chúng trong tập đích $\mathbf{A}$ (với $|\mathbf{A}| = 0.5N$). Khi đó, kỳ vọng sai số nội suy về mặt không gian $\mathbb{E}[\varepsilon_{\text{merge}}(N, r)]$ của các token đã gộp được xác định bởi tỷ lệ giảm r và bị chặn bởi $\mathcal{O}(\Phi(r)N^{-1/2})$, trong đó $\Phi(r)$ là một hàm tăng nghiêm ngặt theo r .*

Chứng minh. Thuật toán gộp token sẽ gộp một token $X_i \in \mathbf{B}$ với láng giềng không gian gần nhất của nó là $X_j \in \mathbf{A}$. Gọi khoảng cách từ X_i đến láng giềng gần nhất này là $D_i = \min_{X_j \in \mathbf{A}} \|X_i - X_j\|_2$.

Vì các điểm phân phối đều, tập hợp $\mathbf{A}$ tạo thành một Poisson point process trong $\mathbb{R}^2$ với mật độ $\rho = |\mathbf{A}| = 0.5N$. Hàm phân phối tích lũy của khoảng cách láng giềng gần nhất D là

$$F(x) = \mathbb{P}(D \leq x) = 1 - \exp(-\pi\rho x^2).$$

Điểm mấu chốt là thuật toán gộp token không gộp tất cả các token trong $\mathbf{B}$. Nó sắp xếp các cạnh và chỉ chọn ra rN cặp có khoảng cách nhỏ nhất. Vì rN token được chọn từ tổng số $0.5N$ token có sẵn, thuật toán thực chất đã cắt đi phân phối tại phân vị $q = \frac{rN}{0.5N} = 2r$.

Khi đó khoảng cách ngưỡng x^* được tìm bằng cách giải phương trình $F(x^*) = 2r$:

$$1 - \exp(-\pi\rho(x^*)^2) = 2r \implies x^* = \sqrt{\frac{-\ln(1-2r)}{\pi\rho}}.$$

Khi đó giá trị kỳ vọng của sai số nội suy của các token được gộp là kỳ vọng có điều kiện của D , với điều kiện $D \leq x^*$:

$$\mathbb{E}[\varepsilon_{\text{merge}}(N, r)] = \frac{1}{P(D \leq x^*)} \int_0^{x^*} x \cdot f(x) dx,$$

trong đó $f(x) = 2\pi\rho x \exp(-\pi\rho x^2)$ là hàm mật độ xác suất. Thay $f(x)$ và $\mathbb{P}(D \leq x^*) = 2r$ vào, ta có:

$$\mathbb{E}[\varepsilon_{\text{merge}}(N, r)] = \frac{1}{2r} \int_0^{x^*} 2\pi\rho x^2 \exp(-\pi\rho x^2) dx.$$

Áp dụng phép đổi biến $u = \pi\rho x^2$, tích phân này trở thành dạng hàm Gamma không đầy đủ cấp thấp (lower incomplete gamma function), $\gamma(s, x) = \int_0^x t^{s-1} e^{-t} dt$:

$$\mathbb{E}[\varepsilon_{\text{merge}}(N, r)] = \frac{1}{2r\sqrt{0.5\pi N}} \gamma(1.5, -\ln(1-2r)).$$

Đặt hệ số phụ thuộc tỷ lệ là $\Phi(r) = \frac{\gamma(1.5, -\ln(1-2r))}{2r\sqrt{0.5\pi}}$, ta thu được:

$$\mathbb{E}[\varepsilon_{\text{merge}}(N, r)] = \Phi(r) N^{-1/2}.$$

Ta cần chứng minh $\Phi(r)$ tăng ngặt trên khoảng $r \in (0, 0.5)$, ta xét hàm số $g(r) = \frac{\gamma(1.5, Z(r))}{2r}$ với $Z(r) = -\ln(1-2r)$. Đạo hàm của $g(r)$ theo quy tắc đạo hàm thương là:

$$g'(r) = \frac{\frac{d}{dr}(\gamma(1.5, Z(r))) \cdot 2r - \gamma(1.5, Z(r)) \cdot 2}{(2r)^2}$$

Áp dụng quy tắc Leibniz cho hàm Gamma không đầy đủ $\gamma(1.5, Z(r))$, ta có:

$$\frac{d}{dr}\gamma(1.5, Z(r)) = \frac{d\gamma}{dZ} \cdot \frac{dZ}{dr} = ((Z(r))^{0.5} e^{-Z}) \cdot \left(\frac{2}{1-2r}\right).$$

Vì $e^{-Z} = e^{\ln(1-2r)} = 1-2r$, biểu thức trên trở thành:

$$\frac{d}{dr}\gamma(1.5, Z(r)) = Z(r)^{0.5}(1-2r) \cdot \frac{2}{1-2r} = 2\sqrt{Z(r)}$$

Thế kết quả này vào biểu thức đạo hàm $g'(r)$, ta thu được:

$$g'(r) = \frac{2\sqrt{Z(r)} \cdot 2r - 2\gamma(1.5, Z(r))}{4r^2} = \frac{2r\sqrt{Z(r)} - \gamma(1.5, Z(r))}{2r^2}$$

Ta cần xét hàm $h(Z) = 2r(Z)\sqrt{Z} - \gamma(1.5, Z)$ theo biến Z . Với $2r = 1 - e^{-Z}$, ta có:

$$h(Z) = (1 - e^{-Z})\sqrt{Z} - \int_0^Z t^{0.5} e^{-t} dt.$$

Lấy đạo hàm $h(Z)$ theo Z :

$$h'(Z) = \left(e^{-Z}\sqrt{Z} + \frac{1 - e^{-Z}}{2\sqrt{Z}} \right) - Z^{0.5} e^{-Z} = \frac{1 - e^{-Z}}{2\sqrt{Z}}$$

Vì $1 - e^{-Z} > 0$ với mọi $Z > 0$, nên $h'(Z) > 0$. Mà khi $r \rightarrow 0$ thì $Z \rightarrow 0$, kéo theo $h(0) \rightarrow 0$. Do hàm $h(Z)$ đồng biến, suy ra $h(Z) > 0$ với mọi $Z > 0$. Điều này dẫn đến $g'(r) > 0$ với mọi $r \in (0, 0.5)$ hay nói cách khác $\Phi(r)$ tăng ngặt. $\square$

16.2.5 Mật độ không gian

Định lý 8 (Định lý BHH [BHH59]). Trong không gian Euclid hai chiều, xét N điểm phân bố đều. Ký hiệu L_N là chiều dài chu trình TSP tối ưu. Khi đó tồn tại hằng số β với $0.625 \leq \beta \leq 0.921$ sao cho

$$\lim_{n \rightarrow \infty} \frac{L_N}{\sqrt{N}} = \beta_2$$

hầu chắc chắn.

Mệnh đề 2. Từ định lý 8, chiều dài chu trình TSP tối ưu có thể được viết dưới dạng $L_N = \beta_2 \sqrt{N} + o(\sqrt{N})$. Khi đó khoảng cách kỳ vọng giữa điểm bất kỳ và họ hàng của nó trong chu trình tối ưu được tính bởi

$$\mathbb{E}[d(x_i, x_{i+1})] = \frac{L_N}{N} = \frac{\beta_2}{\sqrt{N}} + o\left(\frac{1}{\sqrt{N}}\right).$$

Ta thấy rằng, khi kích thước đồ thị mở rộng N lớn, sự gia tăng mật độ không gian khiến khoảng cách kỳ vọng giữa các đỉnh lân cận giảm theo tỷ lệ nghịch với căn bậc hai của N ($\mathcal{O}(1/\sqrt{N})$). Về mặt hình học, điều này ngụ ý rằng trong các đồ thị quy mô lớn, luôn tồn tại các cụm đỉnh nằm sát nhau với đặc trưng định tuyến có độ tương quan cực kỳ cao. Do đó, việc áp dụng cơ chế ghép cặp hai phía để gộp các đỉnh có độ tương đồng cosine lớn sẽ càng nhỏ lại về độ hao hụt thông tin khi N lớn.

Chương 17

Hàm mục tiêu và huấn luyện

Kiến trúc ToMe-POMO được huấn luyện hoàn toàn dựa trên phương pháp học tăng cường thay vì học có giám sát, do đó hệ thống không cần nhãn dữ liệu tối ưu từ các thuật toán chính xác. Tác tử (agent) khám phá đồ thị thông qua policy $p_\theta(\boldsymbol{\pi}|\mathbf{X})$, được tham số hóa bởi trọng số θ của mạng neural. Mục tiêu của hệ thống là tìm ra bộ trọng số θ sao cho kỳ vọng của tổng chiều dài lộ trình $L(\boldsymbol{\pi})$ đạt giá trị cực tiểu:

$$J(\theta) = \mathbb{E}_{\boldsymbol{\pi} \sim p_\theta}[L(\boldsymbol{\pi})].$$

Để tối ưu hóa hàm mục tiêu này, hệ thống áp dụng thuật toán REINFORCE [Bel+16; Kwo+20]. Điểm cốt lõi của phương pháp học POMO là khai thác tính hoán vị của bài toán TSP để sinh ra N lộ trình độc lập $\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_N$ xuất phát từ N đỉnh khác nhau trên cùng một đồ thị $\mathbf{X}$. Lợi dụng đặc tính này, đường cơ sở được tính toán trực tiếp bằng trung bình cộng chiều dài của N lộ trình đó:

$$b_{\mathbf{X}} = \frac{1}{N} \sum_{j=1}^N L(\boldsymbol{\pi}_j).$$

Gradient của hàm mục tiêu sau đó được ước lượng thông qua công thức:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{j=1}^N (L(\boldsymbol{\pi}_j) - b_{\mathbf{X}}) \nabla_\theta \log p_\theta(\boldsymbol{\pi}_j).$$

Đại lượng $(L(\boldsymbol{\pi}_j) - b_{\mathbf{X}})$ đóng vai trò là hàm ưu thế (advantage). Nếu một lộ trình có chiều dài ngắn hơn mức trung bình của toàn bộ đồ thị (chi phí âm), gradient sẽ định hướng bộ tối ưu hóa Adam cập nhật trọng số θ để tăng xác suất chọn lộ trình đó trong tương lai, và ngược lại. Cách tiếp cận này triệt tiêu phương sai của gradient một cách hiệu quả mà không cần huấn luyện thêm một mạng critic phụ trợ [Kwo+20].

Phần IV

Thực nghiệm và đánh giá

Chương 18

Thiết kế thực nghiệm

18.1 Môi trường và phần cứng

Toàn bộ các mô hình và thuật toán đề xuất được lập trình bằng ngôn ngữ Python, sử dụng framework PyTorch để quản lý luồng huấn luyện. Các thành phần học tăng cường và môi trường định tuyến được xây dựng dựa trên thư viện RL4CO. Các thực nghiệm được tiến hành trên nền tảng Google Colab, bộ nhớ RAM 12.7 GB và GPU Google Colab T4 với 15 GB VRAM.

18.2 Xây dựng dữ liệu huấn luyện và kiểm thử

Bài toán TSP được xem xét trên không gian 2 chiều. Tọa độ của các đỉnh được lấy mẫu ngẫu nhiên từ phân phối đều trong không gian hình vuông đơn vị $[0, 1] \times [0, 1]$, tương tự với Kwon và cộng sự [Kwo+20].

Dữ liệu huấn luyện: Để ngăn chặn hiện tượng overfitting, dữ liệu huấn luyện không được sinh sẵn. Thay vào đó, các đồ thị được sinh động ở mỗi mini-batch trong suốt quá trình huấn luyện. Mỗi epoch xử lý tổng cộng 100,000 đồ thị, với kích thước batch bằng 64 [Kwo+20].

Dữ liệu kiểm thử: Để đảm bảo tính công bằng khi đánh giá, một tập dữ liệu kiểm thử tĩnh gồm 10,000 đồ thị được sinh ra và lưu cố định. Mọi mô hình cơ sở và biến thể đều được suy luận trên cùng một tập dữ liệu này.

18.3 Các mô hình cơ sở để so sánh

Để đánh giá hiệu quả của phương pháp ToMe-POMO đề xuất, hai hệ quy chiếu được sử dụng:

- **Concorde:** Một bộ giải chính xác quy chuẩn công nghiệp dựa trên quy hoạch tuyến tính nguyên. Kết quả của Concorde được sử dụng làm cận dưới tối ưu tuyệt đối để tính toán độ sai lệch.
- **POMO [Kwo+20]:** Mô hình học tăng cường sâu gốc chưa tích hợp cơ chế nén đồ thị. Đây là đối tượng so sánh trực tiếp để kiểm định sự đánh đổi giữa việc tiết kiệm tài nguyên và sự hao hụt chất lượng lộ trình do module ToMe gây ra. Tác giả thực hiện lập trình và chạy lại mô hình để cho ra kết quả.

Chương 19

Các chỉ số đánh giá

19.1 Độ lệch tối ưu

Là thước đo quan trọng nhất về chất lượng giải pháp, thể hiện độ chênh lệch phần trăm giữa chiều dài lộ trình do mô hình dự đoán (L_{model}) so với chiều dài tối ưu (L_{opt}). Công thức tính:

$$\text{Gap} = \frac{L_{\text{model}} - L_{\text{opt}}}{L_{\text{opt}}} \times 100\%.$$

Giá trị này càng nhỏ chứng tỏ mô hình sinh ra lộ trình càng gần với mức tối ưu toán học.

19.2 Thời gian suy luận

Đo lường tổng thời gian (tính bằng giây) mà mô hình cần để đưa ra lộ trình giải quyết cho toàn bộ 10,000 đồ thị trong tập kiểm thử. Chỉ số này phản ánh khả năng triển khai của mô hình trong các ứng dụng thời gian thực.

19.3 Thời gian huấn luyện

Thời gian trung bình (tính bằng giây) để mô hình hoàn thành việc học trên 100,000 đồ thị sinh ngẫu nhiên trong một chu kỳ. Phản ánh độ phức tạp tính toán của cấu trúc mạng.

19.4 Mức tiêu thụ bộ nhớ **VRAM**

Dung lượng bộ nhớ **VRAM** cực đại tính bằng gigabytes (GB) được mô hình cấp phát trong quá trình xử lý. Chỉ số này phản ánh trực tiếp độ phức tạp bộ nhớ và khả năng mở rộng của kiến trúc khi đối mặt với các đồ thị có số lượng đỉnh lớn.

Chương 20

Kết quả thực nghiệm chính

Bảng 20.1 trình bày kết quả khảo sát siêu tham số của kiến trúc ToMe-POMO sau 10 epochs huấn luyện trên tập TSP-20. Dữ liệu cho thấy vị trí lớp nén (ℓ) và tỷ lệ nén (r) có ảnh hưởng trực tiếp đến hiệu năng mô hình. Đối với vị trí nén, các biến thể thực hiện cắt tỉa tại lớp sâu hơn ($\ell = 4$) đều ghi nhận chiều dài lộ trình ngắn hơn và độ chênh lệch thấp hơn so với việc cắt tỉa sớm tại lớp 0 hoặc lớp 2 trên mọi tỷ lệ r . Đáng chú ý nhất, tại $r = 0.1$, nén ở lớp 4 đạt độ lệch 0.584% so với baseline, trong khi nén ở lớp 0 bị lệch tới 2.691%. Về mặt tỷ lệ nén, ngưỡng $r = 0.3$ mang lại sự cân bằng ổn định nhất giữa hiệu năng (gap 0.969% tại $\ell = 4$) và lợi ích tài nguyên, do đó cấu hình này được lựa chọn để tiến hành các thực nghiệm hội tụ sâu.

Phương pháp	VRAM (GB)	Train/Ep (s)	Infer (s)	Length	Gap (Concorde)	Gap (POMO)
POMO	0.19	178.19	1.71	3.8716	1.086%	0.000%
<i>ToMe-POMO đề xuất ($r = 0.1$)</i>						
Layer 0	0.26	163.36	1.46	3.9758	3.807%	2.691%
Layer 2	0.27	165.51	1.51	3.9126	2.157%	1.059%
Layer 4	0.28	170.33	1.55	3.8942	1.676%	0.584%
<i>ToMe-POMO đề xuất ($r = 0.3$)</i>						
Layer 0	0.26	168.27	1.45	3.9969	4.358%	3.236%
Layer 2	0.22	182.14	1.42	3.9262	2.512%	1.410%
Layer 4	0.22	185.82	1.69	3.9091	2.065%	0.969%
<i>ToMe-POMO đề xuất ($r = 0.5$)</i>						
Layer 0	0.27	166.05	1.46	4.0411	5.512%	4.378%
Layer 2	0.27	166.44	1.50	3.9681	3.606%	2.493%
Layer 4	0.29	167.32	1.56	3.9393	2.854%	1.749%

Bảng 20.1. Khảo sát siêu tham số trên tập TSP-20 (10 epochs). Đánh giá ảnh hưởng của tỷ lệ nén (r) và vị trí lớp nén (ℓ) đối với mô hình ToMe-POMO. Chiều dài tối ưu tuyệt đối của Concorde là 3.83 [Kwo+20]. Nguồn: Tác giả.

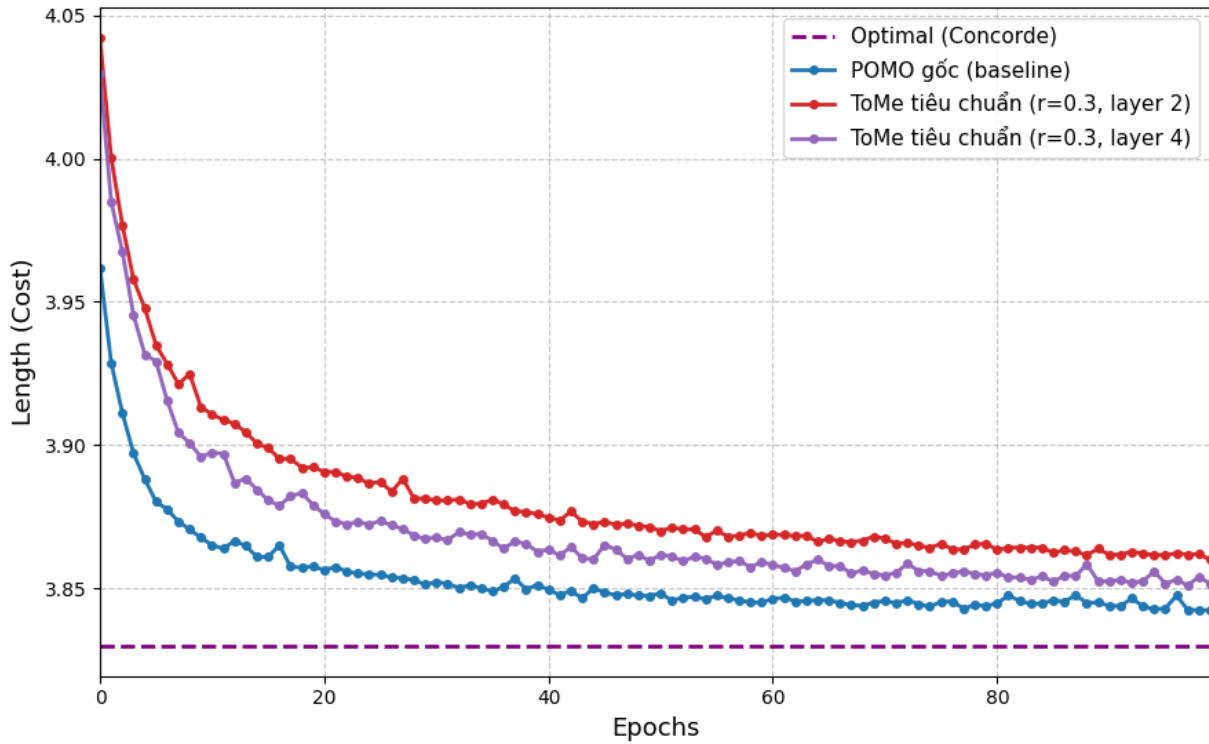
Khi mở rộng quá trình huấn luyện (100 epochs cho TSP-20 và 50 epochs cho TSP-50), bảng 20.2 tổng hợp hiệu suất của các biến thể ToMe-POMO tốt nhất so với mạng nguyên bản. Về chất lượng lộ trình, cả hai cấu hình đề xuất đều bám sát mức tối ưu của hệ quy chiếu. Trên tập TSP-20, mô hình ($r = 0.3, \ell = 4$) đạt chiều dài 3.8540 (độ lệch 0.242%). Trên tập TSP-50 quy mô lớn hơn, biến thể này cũng chỉ duy trì độ lệch ở mức 1.150%.

Về mặt tài nguyên phần cứng, dữ liệu ghi nhận mức tiêu thụ VRAM của các biến thể ToMe-POMO dao

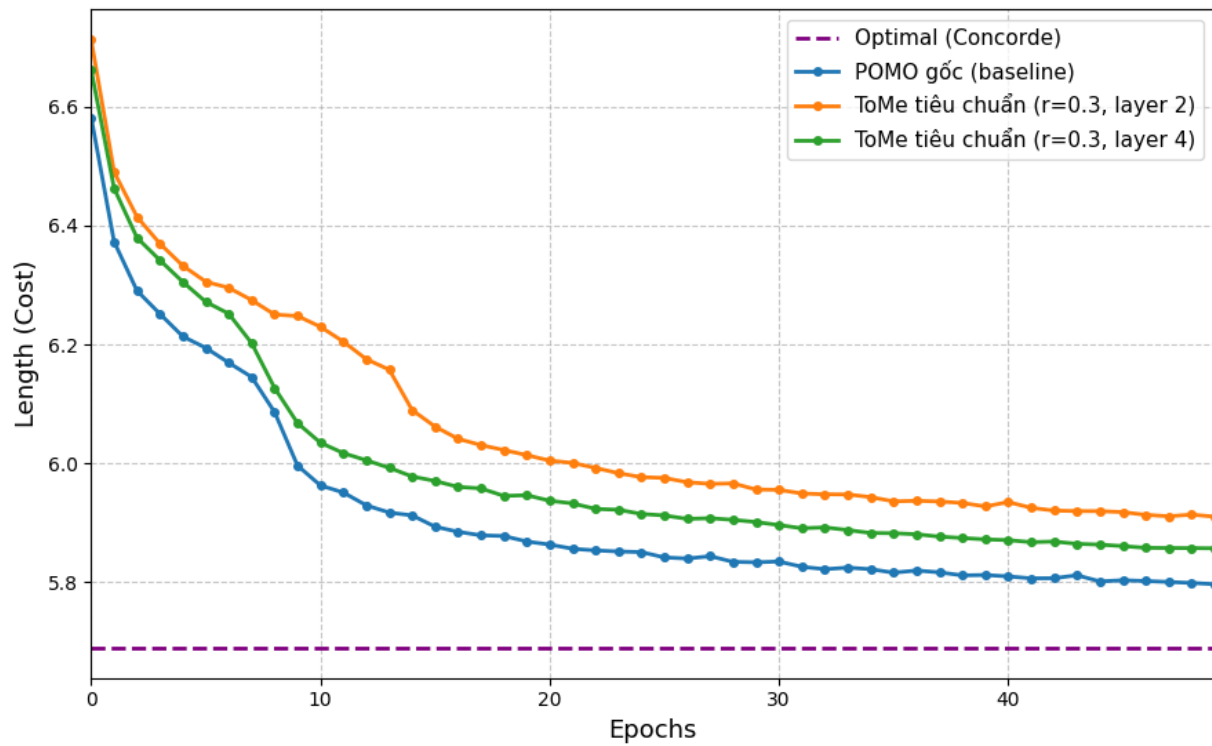
động từ 0.12 đến 0.22 GB ở tập TSP-20, và xung quanh mức 1.06 GB ở tập TSP-50. Mặc dù dung lượng bộ nhớ cấp phát tại không gian đỉnh nhỏ ($N \leq 50$) có sự gia tăng cục bộ so với POMO gốc, thời gian suy luận (inference time) trên tập kiểm thử 10,000 đồ thị lại ghi nhận sự sụt giảm ở cả hai cấu hình: giảm từ 1.56s xuống 1.50s (đối với TSP-20) và từ 5.02s xuống 4.71s (đối với TSP-50). Tiến trình hội tụ chiều dài lộ trình qua từng chu kỳ huấn luyện được trực quan hóa chi tiết tại Hình 20.1 và Hình 20.2.

Method	TSP20				TSP50			
	Len.	Gap	Infer	VRAM	Len.	Gap	Infer	VRAM
POMO	3.8447	0.000%	1.56	0.19	5.8101	0.000%	5.02	0.88
ToMe-POMO $r = 0.3, \ell = 2$	3.8640	0.502%	1.50	0.12	5.9395	2.227%	4.71	1.04
ToMe-POMO $r = 0.3, \ell = 4$	3.8540	0.242%	1.54	0.22	5.8769	1.150%	5.59	1.06

Bảng 20.2. So sánh hiệu năng tổng thể trên TSP-20 và TSP-50. Đánh giá các biến thể ToMe-POMO tối ưu so với POMO nguyên bản sau khi huấn luyện. Độ lệch (gap) được tính tương đối so với POMO. *Nguồn: Tác giả.*



Hình 20.1. Biểu đồ biểu thị độ dài chu trình trên TSP-20 sau 100 epochs. *Nguồn: Tác giả.*



Hình 20.2. Biểu đồ biểu thị độ dài chu trình trên TSP-50 sau 50 epochs. *Nguồn: Tác giả.*

Chương 21

Thảo luận

21.1 Sự bảo toàn đặc trưng của cơ chế nén hai phía

Một trong những thách thức lớn nhất khi áp dụng ToMe vào TSP là sự phá vỡ cấu trúc không gian. Bài toán TSP đòi hỏi độ chính xác tuyệt đối về mặt tọa độ; việc xóa bỏ bất kỳ đỉnh nào thường dẫn đến sự mất phương hướng của tác tử học tăng cường.

Tuy nhiên, kết quả thực nghiệm trên bảng ?? cho thấy mô hình ToMe-POMO chỉ tạo ra một độ lệch cực kỳ nhỏ so với POMO nguyên bản (0.242% trên TSP-20 và 1.150% trên TSP-50). Điều này khẳng định tính đúng đắn của giả thuyết: Việc sử dụng ma trận tương đồng cosine để thiết lập đồ thị hai phía đã ghép cặp thành công các đỉnh có vai trò định tuyến tương đồng nhau. Thay vì loại bỏ hoàn toàn thông tin, phép toán lấy trung bình cộng (average pooling) kết hợp với bước rửa token ở cuối bộ mã hóa đã bảo toàn thành công đặc trưng của đồ thị, giúp bộ giải mã vẫn nhận được đủ N đỉnh mà không bị mất mát thông tin biểu diễn.

21.2 Chi phí phụ trợ ở đồ thị quy mô nhỏ

Dữ liệu thực nghiệm chỉ ra một hiện tượng đáng chú ý: Dù được thiết kế để giảm thiểu bộ nhớ, mức tiêu thụ VRAM của ToMe-POMO tại quy mô $N = 20$ và $N = 50$ lại cao hơn nhẹ so với POMO gốc (tăng từ 0.88 GB lên 1.06 GB ở TSP-50). Đây không phải là một khiếm khuyết của hệ thống. Như đã chứng minh trong định lý 5, chi phí tính toán của kiến trúc bao gồm hai thành phần: chi phí tiết kiệm được từ ma trận attention và chi phí phụ trợ sinh ra từ toán tử ToMe cho top- k và cấp phát zero-tensor khôi phục. Ở các đồ thị có kích thước nhỏ ($N \leq 50$), thành phần chi phí tiết kiệm được từ ma trận attention chưa đủ lớn để chi phối toàn bộ hệ thống. Việc tính toán ma trận attention 50×50 tiêu tốn rất ít VRAM, do đó lượng tài nguyên tiết kiệm không trội được so với chi phí cấp phát bộ nhớ phụ trợ của hàm nén và rửa token.

Tuy nhiên, theo quy luật của hàm đa thức, khi kích thước đồ thị tăng lên các ngưỡng thực tế (ví dụ $N = 500$ hoặc $N = 1000$), chi phí tính toán attention sẽ lớn, gây tràn bộ nhớ. Khi đó, các chi phí phụ trợ tuyến tính của ToMe sẽ trở nên không đáng kể, biến kiến trúc ToMe-POMO trở thành giải pháp khả thi duy nhất để huấn luyện trên các phần cứng có giới hạn.

21.3 Vai trò của độ sâu kiến trúc

Phân tích kết quả từ bảng khảo sát tham số (bảng 20.1) cho thấy việc nén token ở các lớp sâu hơn (lớp $\ell = 4$) mang lại hiệu năng định tuyến tốt hơn hẳn so với việc nén ngay tại các lớp đầu tiên (lớp $\ell = 0$).

Hiện tượng này có thể được giải thích thông qua cơ chế biểu diễn của mạng Transformer. Ở các lớp đầu tiên, các vector đặc trưng đỉnh chỉ mới chứa tọa độ vật lý thô và chưa có đủ thông tin về bối cảnh toàn cục. Nếu thực hiện nén tại đây, đồ thị sẽ gặp phải hiện tượng thất cổ chai thông tin, khiến các siêu đỉnh được tạo ra mang dữ liệu nhiễu và không chính xác. Ngược lại, khi đạt đến lớp 4, các đỉnh đã trải qua nhiều lớp truyền thông điệp qua các cơ chế attention trước đó. Mỗi đỉnh lúc này đã chứa đựng ngữ cảnh toàn cục về vị trí của nó so với toàn đồ thị. Việc gộp token tại giai đoạn này tỏ ra an toàn và hiệu quả hơn rất nhiều.

21.4 Hạn chế của nghiên cứu và hướng phát triển trong tương lai

Mặc dù đã chứng minh được tính đúng đắn về mặt cấu trúc và toán học, nghiên cứu này vẫn còn một số hạn chế mở ra các hướng phát triển tiếp theo.

1. Giới hạn về quy mô thực nghiệm: Do những hạn chế khách quan về giới hạn thời gian huấn luyện và cấu hình phần cứng, các thực nghiệm hiện tại mới chỉ dừng lại ở quy mô $N = 50$. Để chứng minh triệt để sức mạnh tiệm cận của kiến trúc, việc huấn luyện mô hình trên các tập dữ liệu lớn ($N \in [100, 1000]$) là ưu tiên hàng đầu trong tương lai.
2. Tỷ lệ nén động: Hệ thống hiện đang sử dụng một tỷ lệ nén tĩnh r được thiết lập từ đầu (ví dụ $r = 0.3$). Tương lai có thể nghiên cứu tích hợp một mạng con để mô hình có khả năng tự động học và điều chỉnh tỷ lệ r tùy thuộc vào độ phức tạp của từng đồ thị cụ thể trong dữ liệu.
3. **Mở rộng bài toán:** Cấu trúc nén/rã hai phía này hoàn toàn có tiềm năng được tổng quát hóa để áp dụng cho các biến thể định tuyến phức tạp hơn như [bài toán định tuyến xe có giới hạn sức chứa \(CVRP\)](#).

Phần V

Kết luận và định hướng

Chương 22

Kết luận

Khóa luận này đã đề xuất và chứng minh thành công của kiến trúc ToMe-POMO, một hệ thống giải quyết bài toán TSP bằng học tăng cường có tích hợp cơ chế nén đồ thị. Xuất phát từ thách thức về điểm nghẽn bộ nhớ bậc hai $\mathcal{O}(N^2)$ của cơ chế self-attention trong các mô hình Transformer chuẩn, nghiên cứu đã tái cấu trúc bộ mã hóa của POMO [Kwo+20] bằng cách chèn khối gộp token ToMe.

Những đóng góp và kết quả cốt lõi của nghiên cứu bao gồm:

1. Về mặt thuật toán: Thiết kế một cơ chế nén đồ thị không cần tham số học dựa trên nguyên lý ghép cặp hai phía [Bol+23]. Bằng cách khai thác ma trận tương đồng cosine kết hợp với chiến lược lựa chọn tham lam top- k , hệ thống đã thu gọn không gian đặc trưng mà không làm phá vỡ cấu trúc định tuyến của đồ thị. Kỹ thuật rửa token bằng hàm bao đóng (closure) ở cuối bộ mã hóa đã đảm bảo tính tương thích, giúp giữ nguyên vẹn sức mạnh của bộ giải mã POMO gốc.
2. Về mặt thực nghiệm: Khóa luận đã cung cấp các kết quả thực nghiệm rõ ràng trên tập dữ liệu TSP-20 và TSP-50. Kết quả cho thấy ToMe-POMO đạt được sự cân bằng giữa chất lượng và tài nguyên. Mặc dù thu gọn tới 30% lượng token ở các lớp mạng sâu, độ chênh lệch của hệ thống so với POMO nguyên bản được khống chế ở mức cực kỳ thấp (0.242% trên TSP-20 và 1.150% trên TSP-50). Tốc độ suy luận cũng ghi nhận sự sụt giảm tích cực.
3. Về mặt lý thuyết phần cứng: Bằng các định lý toán học về độ phức tạp tiệm cận, nghiên cứu đã chứng minh rằng chi phí phụ trợ của thuật toán nén sẽ không đáng kể hoàn toàn khi số lượng đỉnh N tăng lên, mở ra cơ hội huấn luyện các mô hình định tuyến với cấu hình phần cứng giới hạn.

Nhìn chung, ToMe-POMO không chỉ là một giải pháp tối ưu cục bộ cho bài toán TSP, mà còn là một minh chứng cho thấy sự kết hợp giữa lý thuyết đồ thị truyền thống và kiến trúc phần mềm có thể vượt qua các giới hạn vật lý của phần cứng máy tính.

Chương 23

Định hướng

Kế thừa những nền tảng lý thuyết và thực nghiệm đã được thiết lập, nghiên cứu này mở ra một số hướng phát triển đầy tiềm năng trong tương lai:

1. Thử nghiệm năng lực ngoại suy ở quy mô cực lớn: Điểm mạnh lớn nhất của thiết kế $\mathcal{O}((1-r)^2 N^2)$ là khả năng hoạt động trên các đồ thị khổng lồ. Hướng nghiên cứu tiếp theo sẽ tập trung vào việc thử nghiệm ToMe-POMO trên các tập TSP-500 và TSP-1000, nơi mạng POMO truyền thống sẽ thất bại do tràn bộ nhớ.
2. Cơ chế nén động: Hệ thống hiện hành đang sử dụng siêu tham số tĩnh (như nén cố định 30% ở lớp 4). Một hướng đi tiềm năng là tích hợp cơ chế học tăng cường phụ trợ để mạng có thể tự động quyết định tỷ lệ nén r và vị trí nén ℓ theo thời gian thực, tùy thuộc vào độ phân tán tọa độ của từng đồ thị cụ thể trong batch dữ liệu.
3. Tổng quát hóa cho các biến thể định tuyến đa dạng: Cơ chế gộp và khôi phục đồ thị hai phía hoàn toàn có khả năng được hiệu chỉnh để xử lý các thuộc tính động. Việc ánh xạ kiến trúc ToMe-POMO sang [CVRP](#) hứa hẹn sẽ mang lại những giá trị ứng dụng to lớn cho ngành logistics và vận tải thực tế.

Tài liệu tham khảo

- [App+06] David L Applegate, Robert E Bixby, Vašek Chvátal and William J Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.
- [Bel+16] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi and Samy Bengio. “Neural Combinatorial Optimization with Reinforcement Learning”. in *International Conference on Learning Representations*: (2016).
- [BHH59] Jillian Beardwood, J. H. Halton and J. M. Hammersley. “The shortest path through many points”. in *Mathematical Proceedings of the Cambridge Philosophical Society*: 55.4 (1959), pages 299–327. DOI: [10.1017/S0305004100034095](https://doi.org/10.1017/S0305004100034095).
- [BLP21] Yoshua Bengio, Andrea Lodi and Antoine Prouvost. “Machine learning for combinatorial optimization: a methodological tour d’horizon”. in *European Journal of Operational Research*: 290.2 (2021), pages 405–421. DOI: [10.1016/j.ejor.2020.07.063](https://doi.org/10.1016/j.ejor.2020.07.063).
- [Bol+23] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer and Judy Hoffman. “Token Merging: Your ViT But Faster”. in *International Conference on Learning Representations*: (2023).
- [Chi+19] Rewon Child, Scott Gray, Alec Radford and Ilya Sutskever. “Generating Long Sequences with Sparse Transformers”. in *arXiv preprint arXiv:1904.10509*: (2019).
- [Chr76] Nicos Christofides. “Worst-case analysis of a new heuristic for the travelling salesman problem”. in *Report 388*: (1976).
- [DFJ54] George Dantzig, Ray Fulkerson and Selmer Johnson. “Solution of a Large-Scale Traveling-Salesman Problem”. in *Journal of the Operations Research Society of America*: 2.4 (1954), pages 393–410.
- [FQZ21] Zhang-Hua Fu, Kai-Bin Qiu and Hongyuan Zha. “Generalize a Small Pre-trained Model to Arbitrarily Large TSP Instances”. in *Proceedings of the AAAI Conference on Artificial Intelligence*: volume 35. 8. 2021, pages 7474–7482. DOI: [10.1609/aaai.v35i8.16916](https://doi.org/10.1609/aaai.v35i8.16916).
- [Hel17] Keld Helsgaun. *An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems*. techreport Technical Report. Roskilde, Denmark: Roskilde Universitet, 2017.
- [JLB19] Chaitanya K Joshi, Thomas Laurent and Xavier Bresson. “An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem”. in *arXiv preprint arXiv:1906.01227*: (2019).
- [Koo+22] Wouter Kool, Herke van Hoof, Joaquim Gromicho and Max Welling. “Deep Policy Dynamic Programming for Vehicle Routing Problems”. in *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*: Springer, 2022, pages 190–213.
- [Kor+23] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi and Bryan Catanzaro. “Reducing Activation Recomputation in Large Transformer Models”. in 5: (2023), pages 341–353.
- [KPP22] Minsu Kim, Junyoung Park and Jinkyoo Park. “Sym-NCO: Leveraging Symmetry for Neural Combinatorial Optimization”. in 35: (2022), pages 19340–19352.

- [KS24] Grigory Khromov and Sidak Pal Singh. “Some Fundamental Aspects about Lipschitz Continuity of Neural Networks”. in *International Conference on Learning Representations*: (2024).
- [KV18] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. Springer, 2018.
- [KVV19] Wouter Kool, Herke Van Hoof and Max Welling. “Attention, learn to solve routing problems!” in *International Conference on Learning Representations*: (2019).
- [Kwo+20] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon and Seungjai Min. “POMO: Policy Optimization with Multiple Optima for Reinforcement Learning”. in *International Conference on Neural Information Processing Systems*: (2020).
- [LK73] Shen Lin and Brian W. Kernighan. “An Effective Heuristic Algorithm for the Traveling-Salesman Problem”. in *Operations Research*: 21.2 (1973), pages 498–516. DOI: [10.1287/opre.21.2.498](https://doi.org/10.1287/opre.21.2.498).
- [Nar+21] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee and Matei Zaharia. “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM”. in *International Conference for High Performance Computing, Networking, Storage and Analysis*: (2021).
- [Pop+24] Petrică C. Pop, Ovidiu Cosma, Cosmin Sabo and Corina Pop Sitar. “A Comprehensive Survey on the Generalized Traveling Salesman Problem”. in *European Journal of Operational Research*: 314.3 (2024), pages 819–835. DOI: [10.1016/j.ejor.2023.07.022](https://doi.org/10.1016/j.ejor.2023.07.022).
- [Roy+21] Aurko Roy, Mohammad Saffar, Ashish Vaswani and David Grangier. “Efficient Content-Based Sparse Attention with Routing Transformers”. in *Transactions of the Association for Computational Linguistics*: 9 (2021), pages 53–67. DOI: [10.1162/tac1_a_00353](https://doi.org/10.1162/tac1_a_00353).
- [Sch05] Alexander Schrijver. *On the History of Combinatorial Optimization (till 1960)*. by editor K. Aardal, G.L. Nemhauser and R. Weismantel. Elsevier, 2005.
- [SY23] Zhiqing Sun and Yiming Yang. “DIFUSCO: Graph-based Diffusion Solvers for Combinatorial Optimization”. in 36: (2023), pages 70653–70671. URL: <https://neurips.cc>.
- [Tay+22] Yi Tay, Mostafa Dehghani, Dara Bahri and Donald Metzler. “Efficient Transformers: A Survey”. in *ACM Computing Surveys*: 55.6 (2022). DOI: [10.1145/3530811](https://doi.org/10.1145/3530811).
- [Vas+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser and Illia Polosukhin. “Attention is all you need”. in *International Conference on Neural Information Processing Systems*: 30 (2017).
- [VFJ15] Oriol Vinyals, Meire Fortunato and Navdeep Jaitly. “Pointer Networks”. in *International Conference on Neural Information Processing Systems*: 28 (2015).